Lecture 7: DDPM (Denoising Diffusion Probabilistic Models)

노이즈 추가와 제거를 통한 이미지 생성 — 전체 수식 유도

목차 (Table of Contents)

- 0. 개요 (Overview)
  - 0.1 선수 지식 (Prerequisites)
  - 0.2 학습 목표 (Learning Objectives)
- 1. Forward Process (노이즈 추가)
  - 1.1 한 스텝의 정의
  - 1.2 마르코프 체인
  - 1.3 임의의 t에서의 닫힌 형태 (Closed-Form)
- 2. Reverse Process (노이즈 제거 = 생성)
  - 2.1 목표
  - 2.2 Reverse도 가우시안으로 모델링
  - 2.3 학습 가능한 표적: q(x_{t-1} | x_t, x_0)
  - 2.4 x_0 제거 — 노이즈 예측으로 변환
- 3. ELBO에서 Simple Loss로
  - 3.1 DDPM의 Variational Lower Bound (VLB)
  - 3.2 L_{t-1}의 전개 — 두 가우시안의 KL
  - 3.3 epsilon-prediction으로 변환
  - 3.4 Simple Loss (Ho et al.의 핵심 기여)
- 4. 학습 & 생성 알고리즘
  - 4.1 학습 (Training)
  - 4.2 생성 (Sampling)
- 5. 네트워크 구조: U-Net
  - 5.1 왜 U-Net인가?
  - 5.2 구조
  - 5.3 핵심 구성 요소
- 6. Score-Based Model과의 관계
  - 6.1 수학적 동치
  - 6.2 세 가지 예측 대상의 동치
- 7. DDIM — 빠른 결정론적 샘플링
  - 7.1 DDPM의 문제
  - 7.2 핵심 아이디어
  - 7.3 서브시퀀스 트릭
  - 7.4 DDIM은 ODE 솔버
- 8. 노이즈 스케줄 설계
  - 8.1 선형 스케줄 (DDPM 원논문)
  - 8.2 코사인 스케줄 (Improved DDPM)

    - 8.3 실용적 선택
- 9. 실습 가이드: PyTorch 구현
    - 9.1 NoiseSchedule 클래스
    - 9.2 Time Embedding
    - 9.3 ResBlock (시간 조건부)
    - 9.4 Self-Attention 블록
    - 9.5 U-Net 전체 구조
    - 9.6 Forward Diffusion
    - 9.7 학습 루프 (Training Loop)
    - 9.8 DDPM 샘플링 (생성)
    - 9.9 DDIM 샘플링 (빠른 생성)
- 10. 핵심 수식 요약
- 11. 다음 강의 미리보기
- 12. 참고문헌 (References)

---

0. 개요 (Overview)

> 📄 **논문**: Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020
>
> 이 강의는 현대 생성 모델의 핵심인 DDPM의 수식을 **처음부터 끝까지** 유도한다.

> 💡 **왜 Diffusion Model이 등장했는가?** 2020년 이전까지 이미지 생성의 주류는 **GAN** (Generative Adversarial Network)이었다. GAN은 뛰어난 이미지 품질을 보여주었지만, 치명적인 문제들이 있었다:
>
> - **학습 불안정성**: Generator와 Discriminator의 균형을 맞추기가 극도로 어렵다 (mode collapse, training divergence)
> - **Mode Collapse**: 데이터 분포의 일부 모드만 학습하고 나머지를 무시하는 현상
> - **다양성 부족**: 같은 종류의 이미지만 반복 생성하는 경향
> - **평가 어려움**: likelihood를 계산할 수 없어 모델의 품질을 정량적으로 비교하기 어려움
>
> **VAE** (Variational Autoencoder)는 이론적으로 더 안정적이었지만, 생성 이미지의 품질이 GAN에 비해 크게 떨어졌다 (흐릿한 이미지). 이는 VAE가 단일 스텝에서 잠재 변수 z를 한 번에 이미지 x로 변환해야 하기 때문이다 — 이 변환이 너무 복잡해서 뉴럴넷이 제대로 학습하지 못한다.
>
> **Diffusion Model의 핵심 통찰**: "한 번에 큰 변환을 하지 말고, **아주 작은 변환을 수천 번 반복**하자." 각 스텝에서의 변환은 단순한 가우시안이므로 학습이 쉽고, 이를 충분히 많이 쌓으면 복잡한 데이터 분포도 표현할 수 있다. 이 아이디어는 비평형 통계역학 (non-equilibrium thermodynamics)에서 영감을 받았다 — 물리학에서 복잡한 계(system)도 아주 작은 열역학적 변화를 무한히 반복하면 평형 상태에 도달한다는 원리.

강의 5에서 Score-Based Model의 개념(스코어 함수, Langevin Dynamics)을 배웠다. DDPM은 같은 아이디어를 **실용적으로 구현**한 모델이며, 수학적으로 동치임을 보인다.

DDPM의 역사적 맥락: Diffusion Model의 원형은 Sohl-Dickstein et al. (2015)이 제안했지만, 당시에는 GAN에 비해 성능이 크게 떨어져 주목받지 못했다. 5년 후 Ho et al. (2020)이 핵심적인 단순화 — ϵ-prediction과 Simple Loss — 를 도입하면서 GAN에 필적하는 이미지 품질을 달성했고, 이것이 바로 DDPM이다. 이후 Dhariwal & Nichol (2021)이 "Diffusion Models Beat GANs on Image

Synthesis"라는 제목의 논문에서 Diffusion Model이 GAN을 능가함을 실험적으로 입증하면서, 생성 모델의 패러다임이 완전히 바뀌었다.

구성 요소	내용
Forward Process	깨끗한 이미지에 노이즈를 점진적으로 추가
Reverse Process	노이즈 이미지에서 깨끗한 이미지를 복원 (뉴럴넷 학습)
학습 목표	추가된 노이즈를 예측하는 단순한 MSE Loss
생성	순수 노이즈에서 시작하여 역과정을 반복

> 💡 **비유로 이해하는 DDPM** DDPM의 전체 과정을 하나의 비유로 이해해보자. 깨끗한 이미지를 "완성된 조각상"에 비유하면:
>
> - **Forward Process**: 조각상 위에 모래를 한 줌씩 뿌린다. 1000번 반복하면 조각상은 완전히 모래 더미에 묻혀서 원래 형태를 알 수 없게 된다.
> - **Reverse Process**: 모래 더미에서 모래를 한 줌씩 조심스럽게 걷어낸다. 각 단계에서 "이 위치에서 어느 방향으로 모래를 걷어내야 조각상이 나올까?"를 뉴럴넷이 예측한다.
> - **학습**: 다양한 조각상에 대해 "이만큼 모래가 쌓인 상태에서, 실제로 어떤 모래가 추가되었는가?"를 반복 학습한다.
> - **생성**: 완전히 랜덤한 모래 더미에서 시작하여, 학습된 역과정을 적용하면 새로운 조각상이 나타난다.

0.1 선수 지식 (Prerequisites)

이 강의를 이해하기 위해 다음 개념들에 대한 사전 지식이 필요하다:

분야	필요 개념	관련 강의
확률론	가우시안(정규) 분포, Reparameterization Trick, 독립 가우시안의 합	—
베이즈 정리	사후 확률, 조건부 확률, 가우시안의 곱	—
정보 이론	KL Divergence, 두 가우시안 간의 KL 닫힌 형태	—
VAE	ELBO (Evidence Lower Bound), 변분 추론의 기본 개념	강의 3
Score-Based Model	스코어 함수 $\nabla_x \log p(x)$, Langevin Dynamics, Score Matching	강의 5

> ⚠️ **특히 중요한 것**: 가우시안 분포의 성질 (합, 곱, 조건부)에 대한 이해가 없으면 DDPM의 수식 유도를 따라가기 어렵다. 강의 1-2의 확률론 복습을 권장한다.

핵심 선수 개념 상세 설명

아래는 이 강의에서 반복적으로 사용되는 핵심 개념들의 상세 설명이다. 이미 잘 알고 있다면 건너뛰어도 좋지만, 하나라도 모호하다면 반드시 읽고 넘어가라.

가우시안 (정규) 분포 (Gaussian/Normal Distribution): 확률론에서 가장 중요한 연속 확률 분포로, 평균 μ와 분산 σ^2로 완전히 결정된다. 확률 밀도 함수는 $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$이다. DDPM에서 가우시안이 핵심적인 이유는 세 가지다: (1) 중심극한정리에 의해 많은 독립적 노이즈

의 합은 가우시안에 수렴하므로 노이즈 모델링에 자연스럽다. (2) 두 가우시안의 합, 곱, 조건부 모두 다시 가우시안이므로 수학적으로 다루기 쉽다 (닫힌 형태 유도 가능). (3) 다변량 가우시안 $\mathcal{N}(\mu, \Sigma)$은 고차원에서도 동일한 성질을 유지한다.

Reparameterization Trick: $x \sim \mathcal{N}(\mu, \sigma^2)$에서 샘플링하는 것을 $x = \mu + \sigma \cdot \epsilon,\ \epsilon \sim \mathcal{N}(0, 1)$로 다시 쓰는 기법이다. 왜 필요한가? 원래 "분포에서 샘플링"이라는 연산은 미분 불가능하다 (확률적 노드를 통과하는 그래디언트를 계산할 수 없다). 하지만 reparameterization을 사용하면 확률성이 ϵ으로 분리되고, x는 μ와 σ의 결정론적 함수가 되어 역전파(backpropagation)가 가능해진다. VAE에서 처음 대중화되었고, DDPM에서도 Forward Process를 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$으로 쓸 때 동일한 트릭을 사용한다.

독립 가우시안의 합: $X \sim \mathcal{N}(0, \sigma_1^2)$와 $Y \sim \mathcal{N}(0, \sigma_2^2)$가 **독립**이면, $X + Y \sim \mathcal{N}(0, \sigma_1^2 + \sigma_2^2)$이다. 이것은 DDPM의 Closed-Form 유도에서 가장 핵심적인 성질이다. 증명: 두 독립 확률 변수의 합의 분산은 각 분산의 합이다 ($\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$). 가우시안의 경우, 분산의 합으로 결정되는 새로운 가우시안이 된다. 더 엄밀하게는 적률생성함수(MGF)로 증명할 수 있다: $M_{X+Y}(t) = M_X(t) \cdot M_Y(t) = e^{\sigma_1^2 t^2/2} \cdot e^{\sigma_2^2 t^2/2} = e^{(\sigma_1^2 + \sigma_2^2)t^2/2}$이며, 이것은 $\mathcal{N}(0, \sigma_1^2 + \sigma_2^2)$의 MGF이다.

베이즈 정리 (Bayes' Theorem): 사전 확률(prior)과 우도(likelihood)를 결합하여 사후 확률(posterior)을 구하는 공식이다: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$. DDPM에서는 Reverse Process의 posterior $q(x_{t-1}|x_t, x_0)$를 유도할 때 핵심적으로 사용된다. 특히 중요한 성질: **가우시안 Prior** $\times$ **가우시안 Likelihood = 가우시안 Posterior**. 이것이 성립하는 이유는 두 가우시안의 곱이 지수부의 합이 되고, 이차식 + 이차식 = 이차식이므로 결과도 가우시안이기 때문이다.

KL Divergence (쿨백-라이블러 발산): 두 확률 분포 P와 Q 사이의 "거리"를 측정하는 비대칭적 지표로, $D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_{x\sim P}\left[\log \frac{P(x)}{Q(x)}\right]$이다. 항상 ≥ 0이고, $P = Q$일 때만 0이다. "거리"라고 했지만 대칭이 아니므로 ($D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$) 진정한 거리 함수는 아니다. 직관적 의미: "분포 P에서 샘플을 뽑을 때, 실제 분포가 P인지 Q인지를 구분하는 데 필요한 추가 정보량." DDPM에서는 모델의 역과정 p_θ가 정답 posterior q에 얼마나 가까운지를 측정하는 데 사용된다.

ELBO (Evidence Lower Bound): 관측 데이터의 로그 우도(log-likelihood) $\log p(x)$의 하한(lower bound)이다. 대부분의 생성 모델에서 $\log p(x)$를 직접 계산하는 것은 불가능하다 (잠재 변수를 적분해야 하므로). ELBO는 이 계산 불가능한 양의 최적화 가능한 대리 목적함수(surrogate objective)를 제공한다: $\log p(x) \geq \mathrm{ELBO}$이므로, ELBO를 최대화하면 간접적으로 $\log p(x)$도 최대화하는 효과가 있다. VAE에서 처음 도입되었고, DDPM에서도 동일한 프레임워크를 사용하되 잠재 변수가 T개의 중간 상태 $x_1, \ldots, x_T$가 된다.

0.2 학습 목표 (Learning Objectives)

이 강의를 마치면 다음을 할 수 있어야 한다:

1. **Forward Process의 닫힌 형태**를 유도하고, 왜 $\bar{\alpha}_t$가 핵심인지 설명할 수 있다.
2. **Reverse Process의 posterior** $q(x_{t-1} \mid x_t, x_0)$를 베이즈 정리로 유도하고, 이를 노이즈 예측 형태로 변환할 수 있다.
3. **ELBO에서 Simple Loss**까지의 유도 과정을 이해하고, 왜 가중치를 제거한 Simple Loss가 더 좋은 성능을 보이는지 직관적으로 설명할 수 있다.
4. **DDPM과 Score-Based Model의 수학적 동치**를 보이고, ϵ-prediction과 score의 관계를 기술할 수 있다.
5. **PyTorch로 DDPM을 구현**할 수 있다: Noise Schedule, U-Net, Training Loop, Sampling Loop 포함.

1. Forward Process (노이즈 추가)

> 💡 **왜 노이즈를 "점진적으로" 추가하는가? 한 번에 추가하면 안 되는가?** 이것은 DDPM 설계의 가장 근본적인 질문이다. 만약 깨끗한 이미지 x_0에서 순수 노이즈 $x_T \sim \mathcal{N}(0, I)$로 한 번에 변환한다면, 그 역과정도 한 번에 $\mathcal{N}(0, I)$에서 x_0를 생성해야 한다. 이 단일 스텝의 역변환은 극도로 복잡한 비선형 함수이며, 뉴럴넷이 한 번에 학습하기가 사실상 불가능하다 (이것이 바로 VAE의 "흐릿한 이미지" 문제의 원인이다).
>
> 반면, 노이즈를 **아주 작은 양씩 1000번에 걸쳐** 추가하면, 각 스텝의 역변환은 "아주 약간의 노이즈를 제거하는" 작은 변환이 된다. 핵심 수학적 사실: **충분히 작은 스텝의 Forward Process가 가우시안이면, 그 Reverse Process도 근사적으로 가우시안이다** (Feller, 1949). 따라서 각 역스텝을 단순한 가우시안 $\mathcal{N}(\mu_\theta, \sigma^2 I)$으로 모델링할 수 있고, 뉴럴넷은 평균 μ_θ만 예측하면 된다. 이것이 "점진적 노이즈 추가"의 핵심 동기이다.

1.1 한 스텝의 정의

깨끗한 이미지 x_0에서 시작하여, 매 스텝 t마다 가우시안 노이즈를 조금씩 추가한다:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I)$$

- $\beta_t \in (0, 1)$: **노이즈 스케줄** (타임스텝마다 다른 값)
- $\beta_1 = 0.0001,\ \beta_T = 0.02$ (선형 스케줄, $T = 1000$)
- $\sqrt{1-\beta_t}$: 이전 이미지의 스케일을 살짝 줄이면서
- β_t: 그만큼의 노이즈를 추가

Reparameterization으로 쓰면:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

> 직관: 매 스텝마다 원본 신호를 조금 줄이고($\times\sqrt{1-\beta_t}$), 노이즈를 조금 섞는다($+\sqrt{\beta_t}\,\epsilon$).

> 💡 **왜 하필 $\sqrt{1-\beta_t}$와 $\sqrt{\beta_t}$라는 특정 계수인가?** 이 계수의 선택은 임의적이지 않다 — **분산을 1로 보존**하기 위한 것이다. 만약 x_{t-1}의 분산이 1이라면 (즉, $\mathrm{Var}(x_{t-1}) = 1$):
>
> $$\mathrm{Var}(x_t) = (\sqrt{1-\beta_t})^2 \cdot \mathrm{Var}(x_{t-1}) + (\sqrt{\beta_t})^2 \cdot \mathrm{Var}(\epsilon_t)$$
> $$= (1-\beta_t) \cdot 1 + \beta_t \cdot 1 = 1$$
>
> 따라서 $\mathrm{Var}(x_t) = 1$이 유지된다. 이것은 **분산 보존 (variance-preserving)** 이라 불리며, Forward Process 전체에 걸쳐 데이터의 스케일이 유지됨을 보장한다. 만약 이 조건이 깨지면 — 예컨대 $x_t = x_{t-1} + \sigma\epsilon_t$로 노이즈만 더한다면 — 분산이 스텝마다 증가하여 t가 클 때 x_t의 스케일이 폭발하고, 뉴럴넷이 이를 처리하기 어려워진다. 또한 $T \to \infty$에서 $x_T \sim \mathcal{N}(0, I)$로 수렴하는 것이 보장되어, 생성 시 시작점으로 표준 가우시안을 사용할 수 있다.
>
> 다르게 말하면, $\sqrt{1-\beta_t}$와 $\sqrt{\beta_t}$는 "신호"와 "노이즈"의 비율을 조절하는 믹싱 계수이며, 이 둘의 제곱합이 항상 $(1-\beta_t) + \beta_t = 1$이 되도록 설계된 것이다.

1.2 마르코프 체인

마르코프 체인 (Markov Chain)이란 미래 상태가 오직 현재 상태에만 의존하고 과거 히스토리에는 의존하지 않는 확률 과정(stochastic process)이다. 일상적 비유로 설명하면: 내일 날씨를 예측할 때

오늘 날씨만 보면 되고 어제 날씨는 필요 없다는 가정이다. 수학적으로:

$$P(x_t \mid x_{t-1}, x_{t-2}, \ldots, x_0) = P(x_t \mid x_{t-1})$$

전체 Forward Process는 마르코프 체인:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$$

x_t는 오직 x_{t-1}에만 의존하고, 그 이전 히스토리에는 의존하지 않는다.

> 💡 **왜 마르코프 성질이 중요한가?** DDPM의 Forward Process가 마르코프 성질을 만족하는 이유는 각 스텝에서 노이즈를 **독립적으로** 추가하기 때문이다. x_t는 x_{t-1}에 독립 가우시안 노이즈 ϵ_t를 더해 만들어지므로, x_t를 만드는 데 x_{t-2} 이전의 정보는 필요 없다 — x_{t-1}이 이미 모든 과거 정보를 "요약"하고 있다.
>
> 마르코프 성질이 중요한 실용적 이유는 두 가지다:
>
> 1. **결합 분포의 분해**: $q(x_{1:T} \mid x_0)$를 T개의 조건부 분포의 곱으로 분해할 수 있어, 각 스텝을 독립적으로 분석할 수 있다.
> 2. **Reverse Process의 근사 가능성**: Forward가 마르코프이고 각 스텝이 충분히 작으면, Reverse Process $q(x_{t-1} \mid x_t)$도 근사적으로 가우시안이 되어 뉴럴넷으로 모델링할 수 있다.
>
> 참고로, 후에 배울 DDIM은 이 마르코프 가정을 의도적으로 깨뜨려 (non-Markovian forward process) 더 적은 스텝으로 샘플링할 수 있게 한다.

1.3 임의의 t에서의 닫힌 형태 (Closed-Form)

> 💡 **왜 닫힌 형태(Closed-Form)가 필요한가?** 학습 시 매번 x_0에서 시작하여 t번 반복하여 x_t를 만들어야 한다면, $t = 800$인 샘플 하나를 만드는 데 800번의 노이즈 추가가 필요하다. 배치 크기 64, 에폭 수백 번이면 계산량이 천문학적이다. 닫힌 형태가 있으면 x_0에서 **단 한 번의 연산으로** 임의의 t에 해당하는 x_t를 직접 생성할 수 있다. 이것이 DDPM 학습을 실용적으로 만드는 핵심이다.

$T = 1000$번 반복할 필요 없이, 임의의 t에서 x_t를 **한 번에** 계산할 수 있다.

표기법 정의: $\alpha_t = 1 - \beta_t$ (각 스텝에서 보존되는 신호의 비율), $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ (누적곱으로, t 스텝까지의 **총 신호 보존율**)로 정의하면:

$$\boxed{q(x_t \mid x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I)}$$

즉:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

이 수식의 직관적 의미: x_t는 원본 이미지 x_0의 **스케일된 버전** ($\sqrt{\bar{\alpha}_t} x_0$, 신호 성분)과 **순수 노이즈** ($\sqrt{1 - \bar{\alpha}_t}\epsilon$, 노이즈 성분)의 단순 합이다. $\bar{\alpha}_t$가 1에 가까우면 신호가 지배하고 (깨끗한 이미지), 0에 가까우면 노이즈가 지배한다 (순수 노이즈). 이것은 **Signal-to-Noise Ratio (SNR)**로 해석할 수 있으며, $\mathrm{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$이다.

유도

이 유도의 핵심은 "재귀적 대입 + 독립 가우시안의 합" 두 가지 도구만 사용한다.

배경 지식 — 독립 가우시안의 합: 만약 $A \sim \mathcal{N}(0, \sigma_A^2)$와 $B \sim \mathcal{N}(0, \sigma_B^2)$가 서로 독립이면, $A + B \sim \mathcal{N}(0, \sigma_A^2 + \sigma_B^2)$이다. 직관: 두 독립적인 노이즈 소스를 합치면, 총 "흔들림"(분산)은 각각의 흔들림을 더한 것과 같다. 수학적으로는 분산의 가법성 $\mathrm{Var}(A + B) = \mathrm{Var}(A) + \mathrm{Var}(B)$ (독립일 때 공분산이 0이므로)에 의해 성립하며, 합의 분포가 다시 가우시안이 되는 것은 적률생성함수(MGF)의 곱으로 증명할 수 있다: $M_{A+B}(s) = M_A(s)M_B(s) = e^{\sigma_A^2 s^2/2} e^{\sigma_B^2 s^2/2} = e^{(\sigma_A^2+\sigma_B^2)s^2/2}$.

$t = 1$:

한 스텝 정의에서 $\alpha_1 = 1 - \beta_1$로 치환하면:

$$x_1 = \sqrt{\alpha_1}\, x_0 + \sqrt{1 - \alpha_1}\, \epsilon_1$$

이것은 이미 닫힌 형태이다: $\bar{\alpha}_1 = \alpha_1$이므로 $x_1 = \sqrt{\bar{\alpha}_1} x_0 + \sqrt{1 - \bar{\alpha}_1}\epsilon_1$.

$t = 2$: x_1을 대입하면:

$$x_2 = \sqrt{\alpha_2}\, x_1 + \sqrt{1 - \alpha_2}\, \epsilon_2$$

x_1의 표현식을 대입한다 (재귀적 대입):

$$= \sqrt{\alpha_2}\left(\sqrt{\alpha_1}\, x_0 + \sqrt{1 - \alpha_1}\, \epsilon_1\right) + \sqrt{1 - \alpha_2}\, \epsilon_2$$

분배법칙으로 $\sqrt{\alpha_2}$를 괄호 안에 곱한다:

$$= \sqrt{\alpha_1 \alpha_2}\, x_0 + \underbrace{\sqrt{\alpha_2(1 - \alpha_1)}\, \epsilon_1 + \sqrt{1 - \alpha_2}\, \epsilon_2}_{\text{두 독립 가우시안의 합}}$$

여기서 핵심 관찰: ϵ_1과 ϵ_2는 서로 독립인 표준 가우시안이다. 따라서 $\sqrt{\alpha_2(1 - \alpha_1)}\epsilon_1$은 $\mathcal{N}(0, \alpha_2(1 - \alpha_1))$을 따르고, $\sqrt{1 - \alpha_2}\epsilon_2$는 $\mathcal{N}(0, 1 - \alpha_2)$를 따른다. 독립 가우시안의 합의 성질에 의해, 이 두 항의 합은 분산이 더해진 하나의 가우시안이 된다.

두 독립 가우시안의 합의 분산:

$$\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = \alpha_2 - \alpha_1\alpha_2 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2 = 1 - \bar{\alpha}_2$$

(중간 단계: α_2와 $-\alpha_2$가 상쇄되어 $1 - \alpha_1\alpha_2$만 남는다.)

따라서 두 노이즈 항을 하나의 새로운 가우시안 $\epsilon' \sim \mathcal{N}(0, I)$로 합칠 수 있다:

$$x_2 = \sqrt{\bar{\alpha}_2}\, x_0 + \sqrt{1 - \bar{\alpha}_2}\, \epsilon', \quad \epsilon' \sim \mathcal{N}(0, I)$$

귀납법으로 일반화 (수학적 귀납법):

귀납 가설: $x_k = \sqrt{\bar{\alpha}_k} x_0 + \sqrt{1 - \bar{\alpha}_k}\epsilon'_k$가 성립한다고 가정 ($\epsilon'_k \sim \mathcal{N}(0, I)$).

귀납 단계: $x_{k+1} = \sqrt{\alpha_{k+1}} x_k + \sqrt{1 - \alpha_{k+1}}\epsilon_{k+1}$에 귀납 가설을 대입하면:

$$x_{k+1} = \sqrt{\alpha_{k+1}}\left(\sqrt{\bar{\alpha}_k} x_0 + \sqrt{1 - \bar{\alpha}_k}\epsilon'_k\right) + \sqrt{1 - \alpha_{k+1}}\epsilon_{k+1}$$

$$= \sqrt{\alpha_{k+1}\bar{\alpha}_k} x_0 + \sqrt{\alpha_{k+1}(1 - \bar{\alpha}_k)}\epsilon'_k + \sqrt{1 - \alpha_{k+1}}\epsilon_{k+1}$$

노이즈 항의 합산 분산: $\alpha_{k+1}(1 - \bar{\alpha}_k) + (1 - \alpha_{k+1}) = 1 - \alpha_{k+1}\bar{\alpha}_k = 1 - \bar{\alpha}_{k+1}$

따라서 $x_{k+1} = \sqrt{\bar{\alpha}_{k+1}} x_0 + \sqrt{1 - \bar{\alpha}_{k+1}} \epsilon'_{k+1}$이 성립. QED.

> **핵심 포인트**: 이 닫힌 형태가 학습을 가능하게 만든다. 학습 시 매 iteration마다 랜덤한 t를 뽑고, x_t를 x_0에서 **한 번의 연산**으로 생성할 수 있다. 1000번의 순차적 노이즈 추가가 필요 없다.

노이즈 스케줄의 의미

t	$\bar{\alpha}_t$	$\sqrt{\bar{\alpha}_t}$ **(신호 계수)**	$\sqrt{1 - \bar{\alpha}_t}$ **(노이즈 계수)**	**상태**
0	1.0	1.0	0.0	깨끗한 원본
250	~0.85	~0.92	~0.38	약간 노이즈
500	~0.50	~0.71	~0.71	신호 = 노이즈
750	~0.15	~0.39	~0.92	노이즈 지배적
1000	~0.0	~0.0	~1.0	거의 순수 노이즈

t가 커질수록 원본 신호($\sqrt{\bar{\alpha}_t}$)는 사라지고, 노이즈($\sqrt{1 - \bar{\alpha}_t}$)가 지배한다.

$t = 500$ 근처에서 $\bar{\alpha}_t \approx 0.5$가 되어 신호와 노이즈가 동등해진다 (SNR $\approx$ 1). 이 지점이 "이미지의 대략적 구조는 보이지만 세부 디테일은 완전히 사라진" 상태이다. 뉴럴넷의 관점에서 이 영역이 가장 학습하기 "어려운" 구간이기도 하다 — 너무 깨끗하지도, 너무 노이즈하지도 않아서, 원본 이미지의 구조를 추론해야 하는 가장 애매한 영역이기 때문이다.

---

2. Reverse Process (노이즈 제거 = 생성)

> 💡 **Reverse Process의 핵심 아이디어** Forward Process가 "구조 있는 데이터 → 순수 노이즈"로 가는 길이라면, Reverse Process는 그 길을 거꾸로 걸어가는 것이다. 순수 노이즈 $x_T \sim \mathcal{N}(0, I)$에서 시작하여 한 스텝씩 노이즈를 제거하면, 결국 깨끗한 이미지 x_0에 도달한다. 이것이 바로 "생성"이다. 문제는 이 역방향 길을 정확히 아는 것이 극도로 어렵다는 점이며, DDPM의 모든 수학적 장치는 이 역방향 길을 뉴럴넷으로 근사하기 위한 것이다.

2.1 목표

Forward에서 $x_0 \to x_T$로 노이즈를 추가했으니, Reverse에서 $x_T \to x_0$로 노이즈를 제거하면 **생성**이 된다.

Reverse의 진짜 분포 $q(x_{t-1} \mid x_t)$는 **전체 데이터 분포 $q(x_0)$에 의존**하므로 직접 계산이 불가능하다.

> 💡 **왜 $q(x_{t-1} \mid x_t)$를 직접 계산할 수 없는가? (Intractability)** 베이즈 정리를 적용하면 $q(x_{t-1} \mid x_t) = \frac{q(x_t|x_{t-1})q(x_{t-1})}{q(x_t)}$이다. 여기서 $q(x_t \mid x_{t-1})$은 Forward Process이므로 알고 있지만, 문제는 **marginal 분포** $q(x_{t-1})$과 $q(x_t)$이다. 이들은 전체 데이터 분포에 대한 적분을 포함한다:
>
> $$q(x_t) = \int q(x_t \mid x_0) q(x_0) \, dx_0$$
>
> 이 적분은 **모든 가능한 이미지** x_0에 대해 수행해야 한다. ImageNet의 경우 256x256x3 = 196,608차원 공간의 적분이며, 데이터 분포 $q(x_0)$ 자체를 모르기 때문에 (알면 이미 생성 문제

> 를 풀었을 것) 계산이 불가능하다. 이것은 VAE에서 $p(z \mid x) = p(x \mid z)p(z)/p(x)$의 $p(x)$를 계산할 수 없는 것과 정확히 동일한 intractability 문제이다.

해결: 뉴럴넷 $p_\theta(x_{t-1} \mid x_t)$로 근사한다.

2.2 Reverse도 가우시안으로 모델링

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1};\, \mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t))$$

> 💡 **왜 Reverse를 가우시안으로 모델링할 수 있는가?** 이것은 단순한 가정이 아니라 수학적으로 정당화된 선택이다. Feller (1949)의 결과에 의하면, Forward Process의 각 스텝이 충분히 작으면 ($\beta_t \ll 1$), Reverse Process의 각 스텝 $q(x_{t-1} \mid x_t)$도 가우시안에 근사한다. 직관적으로: 아주 작은 노이즈 변화의 역변환은 아주 작은 역변환이며, 작은 변환은 선형 근사가 유효하고, 선형 변환 + 가우시안 입력 = 가우시안 출력이다. 이것이 $T = 1000$처럼 큰 값을 사용하는 이유이기도 하다 — T가 클수록 각 β_t가 작아지고, 가우시안 근사가 더 정확해진다.

Ho et al.의 핵심 단순화: **분산 Σ_θ는 학습하지 않고 고정**한다.

$$\Sigma_\theta(x_t, t) = \sigma_t^2 I, \quad \sigma_t^2 = \beta_t \quad \text{또는} \quad \sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

> 두 선택 모두 비슷한 성능. β_t는 upper bound, $\tilde{\beta}_t$는 optimal variance.

이 두 값의 의미를 좀 더 자세히 설명하면: $\tilde{\beta}_t$는 x_0를 알 때의 정확한 posterior 분산이고 (아래 2.3에서 유도), β_t는 x_0에 대한 정보 없이 구할 수 있는 상한(upper bound)이다. Nichol & Dhariwal (2021)의 Improved DDPM에서는 이 분산도 학습하여 log-likelihood를 개선했지만, 이미지 품질(FID)에는 큰 차이가 없었다. 이는 이미지 생성에서 **평균(방향)이 분산(불확실성)보다 훨씬 중요**하다는 것을 시사한다.

따라서 **평균 $\mu_\theta(x_t, t)$만 학습**하면 된다.

2.3 학습 가능한 표적: $q(x_{t-1} \mid x_t, x_0)$

$q(x_{t-1} \mid x_t)$는 직접 구할 수 없지만, **x_0를 조건으로 추가하면** 닫힌 형태로 구할 수 있다:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1};\, \tilde{\mu}_t(x_t, x_0),\, \tilde{\beta}_t I)$$

> 💡 **왜 x_0를 조건으로 추가하면 갑자기 계산 가능해지는가?** $q(x_{t-1} \mid x_t)$가 계산 불가능한 이유는 "모든 가능한 원본 이미지"에 대해 적분해야 하기 때문이었다. 하지만 x_0를 **알고 있다고 가정하면**, 노이즈가 어디서 왔는지를 정확히 알게 된다. 구체적으로, x_0와 x_t가 주어지면 Forward Process의 닫힌 형태에 의해 x_t에 추가된 노이즈 ϵ을 역산할 수 있고, 따라서 x_{t-1}의 분포도 정확히 계산할 수 있다. 가우시안 Likelihood ($q(x_t \mid x_{t-1})$) $\times$ 가우시안 Prior ($q(x_{t-1} \mid x_0)$) = 가우시안 Posterior ($q(x_{t-1} \mid x_t, x_0)$)라는 conjugacy 성질 덕분에 결과가 닫힌 형태의 가우시안이 된다.
>
> "학습 시에는 x_0를 알고 있으니 이 posterior를 계산할 수 있고, 생성 시에는 x_0를 모르지만 뉴럴넷이 이를 예측한다" — 이것이 DDPM 학습의 핵심 전략이다.

이것이 우리의 학습 표적이다. p_θ가 이 분포에 가까워지도록 학습한다.

유도 (베이즈 정리 + 가우시안 곱)

베이즈 정리 (Bayes' Theorem)를 적용한다. 베이즈 정리란 관측된 결과(x_t)를 바탕으로 원인(x_{t-1})의 확률을 추론하는 공식이다:

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1}, x_0) \cdot q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}$$

이 식의 각 항의 역할을 명확히 하면:

- $q(x_t \mid x_{t-1}, x_0)$: **Likelihood** — "x_{t-1}이 주어졌을 때 x_t가 관측될 확률"
- $q(x_{t-1} \mid x_0)$: **Prior** — "x_0만 알 때 x_{t-1}에 대한 사전 믿음"
- $q(x_t \mid x_0)$: **Evidence** — x_{t-1}에 무관한 정규화 상수
- $q(x_{t-1} \mid x_t, x_0)$: **Posterior** — 우리가 구하고자 하는 것

마르코프 성질에 의해 $q(x_t \mid x_{t-1}, x_0) = q(x_t \mid x_{t-1})$이므로 ($x_t$는 x_{t-1}만 보면 되고 x_0는 불필요):

$$\propto q(x_t \mid x_{t-1}) \cdot q(x_{t-1} \mid x_0)$$

($q(x_t \mid x_0)$는 x_{t-1}에 의존하지 않으므로 정규화 상수로 취급하여 $\propto$로 표기한다.)

각각을 가우시안으로 쓰면 (Section 1에서 유도한 결과를 그대로 사용):

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{\alpha_t}\, x_{t-1},\ \beta_t I)$$

$$q(x_{t-1} \mid x_0) = \mathcal{N}(x_{t-1};\ \sqrt{\bar{\alpha}_{t-1}}\, x_0,\ (1-\bar{\alpha}_{t-1}) I)$$

두 가우시안의 곱은 x_{t-1}에 대해 다시 가우시안이다. 이것은 가우시안 분포의 핵심적인 **conjugacy** 성질이며, DDPM의 수학적 우아함의 근원이다. 두 가우시안의 곱이 왜 가우시안인지 직관적으로 설명하면: 가우시안의 확률 밀도는 $\exp(-\text{이차식})$ 형태이고, 두 가우시안의 곱은 $\exp(-\text{이차식}_1) \times \exp(-\text{이차식}_2) = \exp(-(\text{이차식}_1 + \text{이차식}_2))$이다. 이차식 + 이차식 = 이차식이므로, 결과도 $\exp(-\text{이차식})$ 형태, 즉 가우시안이다.

지수부를 전개하여 x_{t-1}에 대해 정리하면:

지수부 $\propto -\frac{1}{2}\left[\frac{(x_t - \sqrt{\alpha_t} x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0)^2}{1-\bar{\alpha}_{t-1}}\right]$

상세 제곱 완성 (Completing the Square)

제곱 완성이란 $ax^2 + bx + c$ 형태의 이차식을 $a(x-h)^2 + k$ 형태로 변환하여 가우시안의 평균과 분산을 읽어내는 기법이다. 이 기법은 DDPM 유도에서 가장 중요한 대수적 도구이다.

Step 1: 각 가우시안의 지수부를 x_{t-1}에 대해 전개한다.

첫 번째 항 ($q(x_t \mid x_{t-1})$의 지수부):

$$-\frac{(x_t - \sqrt{\alpha_t} x_{t-1})^2}{2\beta_t} = -\frac{x_t^2 - 2\sqrt{\alpha_t} x_t x_{t-1} + \alpha_t x_{t-1}^2}{2\beta_t}$$

두 번째 항 ($q(x_{t-1} \mid x_0)$의 지수부):

$$-\frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0)^2}{2(1-\bar{\alpha}_{t-1})} = -\frac{x_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}} x_0 x_{t-1} + \bar{\alpha}_{t-1} x_0^2}{2(1-\bar{\alpha}_{t-1})}$$

Step 2: x_{t-1}에 대한 항만 모은다 (x_{t-1}에 무관한 항은 정규화 상수에 흡수).

x_{t-1}^2의 계수:

$$-\frac{1}{2}\left(\frac{\alpha_t}{\beta_t}+\frac{1}{1-\bar{\alpha}_{t-1}}\right)=-\frac{1}{2}\cdot\frac{\alpha_t(1-\bar{\alpha}_{t-1})+\beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}$$

여기서 분자를 정리한다: $\alpha_t(1-\bar{\alpha}_{t-1})+\beta_t=\alpha_t-\alpha_t\bar{\alpha}_{t-1}+1-\alpha_t=1-\alpha_t\bar{\alpha}_{t-1}=1-\bar{\alpha}_t$ (왜냐하면 $\bar{\alpha}_t=\alpha_t\bar{\alpha}_{t-1}$이고 $\beta_t=1-\alpha_t$이므로).

따라서 x_{t-1}^2의 계수: $-\frac{1}{2}\cdot\frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})}$

이로부터 **posterior 분산**을 읽어낸다 (가우시안 $\exp(-\frac{x^2}{2\sigma^2})$에서 x^2의 계수가 $-\frac{1}{2\sigma^2}$):

$$\frac{1}{\tilde{\beta}_t}=\frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})}\quad\Rightarrow\quad\boxed{\tilde{\beta}_t=\frac{(1-\bar{\alpha}_{t-1})\beta_t}{1-\bar{\alpha}_t}}$$

Step 3: x_{t-1}의 1차 항의 계수를 모아 평균을 구한다.

x_{t-1}의 계수:

$$\frac{\sqrt{\alpha_t}}{\beta_t}x_t+\frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}x_0$$

가우시안 $\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)=\exp\left(-\frac{x^2}{2\sigma^2}+\frac{\mu x}{\sigma^2}-\frac{\mu^2}{2\sigma^2}\right)$에서 x의 계수가 $\frac{\mu}{\sigma^2}$이므로:

$$\frac{\tilde{\mu}_t}{\tilde{\beta}_t}=\frac{\sqrt{\alpha_t}}{\beta_t}x_t+\frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}x_0$$

$$\tilde{\mu}_t=\tilde{\beta}_t\left(\frac{\sqrt{\alpha_t}}{\beta_t}x_t+\frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}x_0\right)$$

$\tilde{\beta}_t=\frac{(1-\bar{\alpha}_{t-1})\beta_t}{1-\bar{\alpha}_t}$를 대입하면:

$$\boxed{\tilde{\mu}_t(x_t,x_0)=\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0+\frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t}$$

> **검산**: $\tilde{\mu}_t$에서 x_0와 x_t의 계수의 합이 의미 있는지 확인해보자. $x_0=x_t$일 때 (노이즈가 없는 경우), $\tilde{\mu}_t=\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t+\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_0$이어야 한다. 이 계수가 $\sqrt{\bar{\alpha}_{t-1}}$에 가까울수록 올바른 것이다 (왜냐하면 x_{t-1}의 기대값은 $\sqrt{\bar{\alpha}_{t-1}}x_0$이어야 하므로).

2.4 x_0 제거 — 노이즈 예측으로 변환

> 💡 **왜 x_0를 제거해야 하는가?** 위에서 유도한 posterior 평균 $\tilde{\mu}_t(x_t,x_0)$에는 x_0가 들어 있다. 학습 시에는 x_0 (원본 이미지)를 알고 있으므로 문제가 없지만, **생성 시에는 x_0를 모른다** — x_0를 만들어내는 것이 목표이니까! 따라서 $\tilde{\mu}_t$를 x_0 없이, 즉 x_t와 뉴럴넷의 예측만으로 표현해야 한다. 여기서 Ho et al.의 핵심적인 통찰이 등장한다: Forward Process의 닫힌 형태를 이용해 x_0를 ϵ (추가된 노이즈)으로 치환하고, 뉴럴넷이 이 ϵ을 예측하게 하는 것이다.

$\tilde{\mu}_t$에 x_0가 포함되어 있는데, 생성 시에는 x_0를 모른다. 어떻게 하는가?

Step 1: Forward Process의 닫힌 형태 $x_t=\sqrt{\bar{\alpha}_t}\,x_0+\sqrt{1-\bar{\alpha}_t}\,\epsilon$에서 x_0를 역으로 풀면:

$$x_0=\frac{1}{\sqrt{\bar{\alpha}_t}}\left(x_t-\sqrt{1-\bar{\alpha}_t}\,\epsilon\right)$$

직관: "x_t에서 노이즈 성분 $\sqrt{1-\bar{\alpha}_t}\epsilon$을 빼고, 스케일을 복원하면 원본 x_0를 되찾을 수 있다."

Step 2: 이것을 $\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$에 대입한다.

x_0에 위의 표현을 대입:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

Step 3: x_t 항을 모은다.

$$= \left(\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\right)x_t - \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} \cdot \sqrt{1-\bar{\alpha}_t} \cdot \epsilon$$

x_t의 계수를 정리한다. $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$이므로 $\sqrt{\bar{\alpha}_{t-1}}/\sqrt{\bar{\alpha}_t} = 1/\sqrt{\alpha_t}$:

$$x_t\text{의 계수} = \frac{\beta_t}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} = \frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}}$$

분자: $\beta_t + \alpha_t - \alpha_t\bar{\alpha}_{t-1} = (1-\alpha_t) + \alpha_t - \bar{\alpha}_t = 1 - \bar{\alpha}_t$

따라서 x_t의 계수 $= \frac{1-\bar{\alpha}_t}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}} = \frac{1}{\sqrt{\alpha_t}}$

ϵ의 계수를 정리하면: $\frac{\beta_t}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}} \cdot \sqrt{1-\bar{\alpha}_t} = \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}$

최종 결과:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$$

> **핵심**: 평균 $\tilde{\mu}_t$는 x_t와 **실제로 추가된 노이즈 ϵ**으로만 표현된다.

이 결과의 직관적 의미: Reverse Process의 한 스텝은 (1) 현재 노이즈 이미지 x_t에서 시작하여, (2) 그 안에 포함된 노이즈 ϵ의 일부분($\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$만큼)을 빼고, (3) 전체를 $\frac{1}{\sqrt{\alpha_t}}$로 스케일하는 것이다. 마치 "노이즈를 조금 제거하고 신호를 복원한다"는 직관과 정확히 일치한다.

뉴럴넷 $\epsilon_\theta(x_t, t)$가 ϵ을 예측하면:

$$\boxed{\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)}$$

> 💡 **왜 x_0를 직접 예측하지 않고 ϵ을 예측하는가?** 사실 수학적으로는 x_0를 예측하든 ϵ을 예측하든 동치이다 ($x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$로 서로 변환 가능). 그러나 Ho et al.은 실험에서 ϵ-prediction이 x_0-prediction보다 더 좋은 이미지 품질을 보임을 발견했다. 직관적 이유: (1) 노이즈 $\epsilon \sim \mathcal{N}(0, I)$은 모든 t에서 동일한 분포를 따르므로, 뉴럴넷의 출력 스케일이 t에 무관하게 안정적이다. 반면 x_0를 직접 예측하면 t가 클 때 (노이즈가 심할 때) 예측이 매우 불안정해진다. (2) ϵ-prediction은 Score-Based Model의 Score Matching과 수학적으로 동치이며 (Section 6 참조), 이론적으로도 더 자연스러운 학습 목표이다.

---

3. ELBO에서 Simple Loss로

> 💡 **왜 ELBO가 필요한가?** 생성 모델의 궁극적 목표는 데이터의 로그 우도 $\log p_\theta(x_0)$를 최대화하는 것이다. 즉, 모델이 실제 데이터를 높은 확률로 생성하도록 학습한다. 그런데 DDPM에서 $p_\theta(x_0) = \int p_\theta(x_{0:T}) dx_{1:T}$이며, 이 적분은 T개의 잠재 변수 $x_1, \ldots, x_T$ 전체에 대한 것이므로 직접 계산이 불가능하다. ELBO (Evidence Lower Bound)는 이 계산 불가능한 $\log p_\theta(x_0)$의 하한을 제공하여, 하한을 최대화함으로써 간접적으로 로그 우도를 최대화할 수 있게 한다. 이것은 VAE에서 사용한 것과 동일한 변분 추론(Variational Inference) 프레임워크이다.

3.1 DDPM의 Variational Lower Bound (VLB)

VAE와 마찬가지로, $\log p(x_0)$에 대한 하한을 유도한다.

ELBO 유도의 상세 과정

출발점: $\log p_\theta(x_0)$를 직접 다루기 위해 **Jensen's Inequality**를 사용한다. Jensen's Inequality란 볼록 함수(convex function) f에 대해 $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$이 성립하는 부등식이다. $\log$는 오목 함수(concave function)이므로 부등호가 반대로 적용된다: $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$.

Step 1: 잠재 변수 $x_{1:T}$에 대한 적분을 명시적으로 쓴다.

$$\log p_\theta(x_0) = \log \int p_\theta(x_{0:T})\, dx_{1:T}$$

Step 2: Forward Process $q(x_{1:T} \mid x_0)$를 importance sampling 분포로 도입한다 (곱하고 나누기).

$$= \log \int \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} q(x_{1:T} \mid x_0)\, dx_{1:T} = \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \right]$$

Step 3: Jensen's Inequality를 적용하여 $\log$를 기대값 안으로 넣는다 ($\log$가 오목함수이므로 $\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log \cdot]$).

$$\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \right] = \text{ELBO}$$

이것이 **Evidence Lower Bound (ELBO)**이다. "Evidence"란 관측 데이터 x_0의 확률 $p_\theta(x_0)$를 의미하며, ELBO는 그 로그값의 하한이다.

Step 4: ELBO를 분해한다. $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$이고, $q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$이므로:

$$\text{ELBO} = \mathbb{E}_q \left[\log p(x_T) + \sum_{t=1}^{T} \log \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_t \mid x_{t-1})} \right]$$

조건부 확률의 관계를 이용하여 $q(x_t \mid x_{t-1})$을 $q(x_{t-1} \mid x_t, x_0)$와 연결하는 텔레스코핑(telescoping) 기법을 적용하면, 최종적으로 세 항으로 정리된다:

$$\text{VLB} = \underbrace{-D_{\text{KL}}(q(x_T \mid x_0) \| p(x_T))}_{L_T\text{: Prior matching}} + \sum_{t=2}^{T} \underbrace{-D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}\text{: Denoising matching}} + \underbrace{\log p_\theta(x_0 \mid x_1)}_{L_0\text{: Reconstruction}}$$

각 항의 의미

항	의미	학습 가능?

L_T	Forward 끝이 $\mathcal{N}(0, I)$에 가까운지	아니오 — β 스케줄이 결정, 상수
L_{t-1}	각 스텝의 역과정이 정답에 가까운지	예 — 핵심 학습 대상
L_0	최종 복원 품질	예

L_T에 대한 추가 설명: $q(x_T \mid x_0) \approx \mathcal{N}(0, I)$이 되도록 β 스케줄이 설계되었으므로 (Section 1.3의 표에서 $\bar{\alpha}_{1000} \approx 0$), $L_T \approx 0$이 된다. 이 항은 학습할 파라미터가 없으므로 무시한다.

L_0에 대한 추가 설명: 마지막 스텝에서 x_1으로부터 x_0를 복원하는 항이다. 이산 이미지 데이터 (0-255 픽셀 값)의 경우 이 항의 처리가 약간 다를 수 있지만 (가우시안 CDF를 사용한 이산화), 전체 VLB에서 차지하는 비중이 작아 실제로는 큰 영향이 없다.

L_{t-1}이 핵심이다: $T-1$개의 항 ($t = 2, \ldots, T$)으로 구성되며, 각 항은 "정답 Reverse 분포 $q(x_{t-1} \mid x_t, x_0)$와 모델의 Reverse 분포 $p_\theta(x_{t-1} \mid x_t)$가 얼마나 가까운가"를 KL Divergence로 측정한다.

3.2 L_{t-1}의 전개 — 두 가우시안의 KL

$q(x_{t-1} \mid x_t, x_0)$와 $p_\theta(x_{t-1} \mid x_t)$ 모두 가우시안이고 분산이 같으므로($\tilde{\beta}_t = \sigma_t^2$):

$$D_{\mathrm{KL}}(q \| p_\theta) = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2$$

> **분산이 같은 두 가우시안의 KL은 평균 사이의 거리의 제곱에 비례.** 가장 단순한 형태.

두 가우시안 간 KL Divergence의 일반 공식과 유도

일반적으로 두 가우시안 $P = \mathcal{N}(\mu_1, \sigma_1^2)$와 $Q = \mathcal{N}(\mu_2, \sigma_2^2)$ 간의 KL Divergence는:

$$D_{\mathrm{KL}}(P \| Q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

이 공식의 유도: KL Divergence의 정의 $D_{\mathrm{KL}}(P \| Q) = \mathbb{E}_{x \sim P}[\log P(x) - \log Q(x)]$에서 시작한다.

$$\log P(x) = -\frac{1}{2} \log(2\pi\sigma_1^2) - \frac{(x - \mu_1)^2}{2\sigma_1^2}$$

$$\log Q(x) = -\frac{1}{2} \log(2\pi\sigma_2^2) - \frac{(x - \mu_2)^2}{2\sigma_2^2}$$

$$D_{\mathrm{KL}} = \mathbb{E}_P \left[\frac{1}{2} \log \frac{\sigma_2^2}{\sigma_1^2} + \frac{(x - \mu_2)^2}{2\sigma_2^2} - \frac{(x - \mu_1)^2}{2\sigma_1^2} \right]$$

$\mathbb{E}_P[(x - \mu_1)^2] = \sigma_1^2$ (분산의 정의)이고, $\mathbb{E}_P[(x - \mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$ (x의 분산 + 편향의 제곱)이므로:

$$= \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

DDPM의 경우 $\sigma_1 = \sigma_2$ (두 분포의 분산이 동일)이므로:

$$D_{\mathrm{KL}} = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} = 0 + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_1^2} - \frac{1}{2} = \frac{(\mu_1 - \mu_2)^2}{2\sigma_1^2}$$

따라서 **분산이 같을 때 KL은 순수하게 평균 차이의 제곱**이 된다. 이것이 DDPM의 Loss가 단순한 MSE로 귀결되는 수학적 이유이다.

3.3 ϵ-prediction으로 변환

$\tilde{\mu}_t$와 μ_θ 모두 같은 형태이므로:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right), \quad \mu_\theta = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$$

차이를 계산하면:

$$\tilde{\mu}_t - \mu_\theta = \frac{1}{\sqrt{\alpha_t}} \cdot \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}(\epsilon_\theta(x_t, t) - \epsilon)$$

(두 식에서 $\frac{x_t}{\sqrt{\alpha_t}}$ 항은 동일하므로 상쇄된다. 남는 것은 ϵ과 ϵ_θ의 차이에 계수가 곱해진 것뿐이다.)

따라서:

$$\|\tilde{\mu}_t - \mu_\theta\|^2 = \frac{\beta_t^2}{\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

L_{t-1}에 대입하면:

$$L_{t-1} = \frac{1}{2\sigma_t^2} \cdot \frac{\beta_t^2}{\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(x_t, t)\|^2 = \underbrace{\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}}_{w_t\text{: 타임스텝 의존 가중치}} \|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

앞의 계수 w_t는 t에만 의존하는 상수 (학습 파라미터 θ와 무관). 따라서:

$$L_{t-1} \propto \|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

3.4 Simple Loss (Ho et al.의 핵심 기여)

계수 w_t까지 제거하면:

$$\boxed{\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim U[1,T],\, x_0\sim q(x_0),\, \epsilon\sim\mathcal{N}(0,I)}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]}$$

여기서 $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$.

이 Loss의 놀라운 단순함: 결국 DDPM의 학습은 **(1) 원본 이미지에 랜덤 노이즈를 추가하고, (2) 뉴럴넷이 그 노이즈를 맞추고, (3) MSE로 비교**하는 것이 전부이다. VAE의 ELBO, KL Divergence 항, reconstruction 항 등 복잡한 구조가 이 단순한 MSE 하나로 귀결된다는 것이 DDPM의 수학적 아름다움이다.

> Ho et al.은 계수를 제거한 Simple Loss가 계수를 유지한 VLB보다 **실험적으로 더 좋은 성능**을 보임을 발견했다. 이유: 낮은 t (디테일)에 더 큰 가중치를 부여하는 효과.

> ⚠️ **왜 계수를 제거하면 더 좋은가?** VLB의 정확한 계수 $w_t = \frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$는 t가 작을 때(이미지가 깨끗할 때) 매우 작은 값을 가진다. 이는 디테일 복원 학습에 불리하다. 계수를 제거하면 모든 t에 균등한 가중치가 부여되어, 특히 디테일이 중요한 낮은 t에서의 학습이 강화된다.

이를 좀 더 자세히 분석하면:

- **큰 t (노이즈가 지배적)**: 원본 이미지의 구조가 거의 사라진 상태이다. 뉴럴넷은 대략적인 전역 구조 (큰 형태, 색상 분포)만 예측하면 된다. 이 영역에서 w_t가 크다는 것은 "쉬운 문제에 높은 가중치를 부여"하는 것과 같다.
- **작은 t (이미지가 거의 깨끗)**: 원본 이미지와 거의 같지만 미세한 노이즈가 있는 상태이다. 뉴럴넷은 미세한 디테일 (텍스처, 가장자리, 미세한 색상 변화)을 정확히 복원해야 한다. 이 영역에서 w_t가 작다는 것은 "어렵고 중요한 문제에 낮은 가중치를 부여"하는 것이다.
- **Simple Loss ($w_t = 1$)**: 모든 t를 균등하게 다루므로, 작은 t에서의 디테일 학습이 상대적으로 강화된다. 결과적으로 생성 이미지의 디테일과 선명도가 크게 향상된다.

이론적 관점에서 Simple Loss는 더 이상 정확한 VLB가 아니므로 log-likelihood의 하한을 최적화하는 것이 아니다. 그러나 FID (이미지 품질 지표)로 측정했을 때 월등히 좋은 결과를 보여준다. 이는 "이론적으로 최적인 목적함수"와 "실용적으로 최선인 목적함수"가 다를 수 있음을 보여주는 좋은 사례이다.

---

4. 학습 & 생성 알고리즘

> 💡 **학습과 생성의 비대칭성** DDPM의 학습과 생성은 구조적으로 매우 다르다. **학습**은 닫힌 형태 덕분에 단일 스텝으로 이루어지지만 (임의의 t에 대해 x_t를 한 번에 생성), **생성**은 T개의 스텝을 순차적으로 거쳐야 한다. 이 비대칭성은 DDPM의 가장 큰 장점이자 단점이다: 학습은 매우 효율적이지만, 생성은 느리다.

4.1 학습 (Training)

```
Algorithm 1: Training
━━━━━━━━━━━━━━━━━━━━━━

repeat
    x_0 ~ q(x_0)                                # 훈련 데이터에서 샘플
    t ~ Uniform({1, ..., T})                    # 랜덤 타임스텝
    epsilon ~ N(0, I)                                 # 랜덤 노이즈
    x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1-alpha_bar_t) * epsilon         # 
    Loss = ||epsilon - epsilon_theta(x_t, t)||^2               # 실제 노이즈 
    theta <- theta - eta * grad_theta(Loss)                     # 경사 하강법
until converged
```

핵심 포인트:

- 매 배치마다 t를 **랜덤 선택** → 모든 타임스텝을 고르게 학습
- x_t를 x_0에서 **한 번에** 생성 (1000번 반복 불필요)
- Loss는 단순한 **MSE** (실제 노이즈 ϵ vs 예측 ϵ_θ)

알고리즘의 각 줄을 상세히 해석하면:

1. $x_0 \sim q(x_0)$: 학습 데이터셋에서 이미지 하나를 가져온다. 이것이 "정답"이 된다.
2. $t \sim \text{Uniform}(\{1, \ldots, T\})$: 1에서 1000 사이의 정수를 균일하게 랜덤 선택한다. 이 t가 "이 샘플에서 어느 정도의 노이즈를 추가할 것인가"를 결정한다. 균일 분포를 사용하는 이유는 모든 t를 고르게 학습하기 위해서이다 (Simple Loss에서 $w_t = 1$으로 설정한 것과 일관).
3. $\epsilon \sim \mathcal{N}(0, I)$: 표준 가우시안에서 노이즈를 하나 샘플링한다. 이것이 "정답 노이즈"가 된다 — 뉴럴넷이 이 ϵ을 맞추는 것이 학습 목표이다.
4. x_t **계산**: Section 1.3의 닫힌 형태를 사용하여, x_0에서 t 스텝 후의 노이즈 이미지를 **단 한 번의 연산으로** 생성한다. 이것이 뉴럴넷의 입력이 된다.

5. **Loss 계산**: 뉴럴넷 $\epsilon_\theta(x_t, t)$가 x_t와 t를 받아 노이즈를 예측하고, 실제 노이즈 ϵ과의 MSE를 계산한다.
6. **파라미터 업데이트**: 표준 경사 하강법으로 θ를 업데이트한다.

4.2 생성 (Sampling)

```
Algorithm 2: Sampling
──────────────────────

x_T ~ N(0, I)                                   # 순수 노이즈에서 시작
for t = T, T-1, ..., 1 do
    z ~ N(0, I) if t > 1, else z = 0            # 마지막 스텝에서는 노이즈 안 더함
    x_{t-1} = 1/sqrt(alpha_t) * (x_t - beta_t/sqrt(1-alpha_bar_t) * epsilon_t
done
return x_0
```

각 스텝을 분해하면:

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)}_{\text{예측 평균 } \mu_\theta} + \underbrace{\sigma_t \cdot z}_{\text{노이즈 주입}}$$

- 예측 평균: "예측한 노이즈를 빼서 한 스텝 깨끗하게"
- 노이즈 주입: 확률적 다양성 유지 (SDE의 확산 항)

> 💡 **왜 마지막 스텝 ($t = 1$)에서는 노이즈를 더하지 않는가?** 마지막 스텝에서 $z = 0$으로 설정하는 이유: $t = 1 \to t = 0$은 "거의 깨끗한 이미지에서 완전히 깨끗한 이미지로"의 전환이다. 여기서 추가 노이즈를 주입하면 최종 출력에 불필요한 노이즈가 남게 된다. 중간 스텝에서의 노이즈 주입은 Reverse Process가 확률적으로 다양한 경로를 탐색하게 하여 mode collapse를 방지하지만, 마지막 스텝에서는 최종 품질을 위해 노이즈를 제거한다.

> 💡 **왜 생성 시에도 노이즈를 추가하는가? ($\sigma_t z$ 항)** 직관적으로 이해하기 어려운 부분이다 — "노이즈를 제거하는 과정인데 왜 노이즈를 또 추가하는가?" 이유: Reverse Process는 **확률적** 과정이다. Forward Process에서 $x_{t-1} \to x_t$로 갈 때 노이즈가 추가되었듯이, 역으로 $x_t \to x_{t-1}$로 갈 때도 적절한 확률적 변동이 있어야 한다. SDE (Stochastic Differential Equation) 관점에서 이 노이즈 항은 역방향 SDE의 확산(diffusion) 항에 해당한다. 이 노이즈가 없으면 (즉, $\sigma_t = 0$이면) DDIM이 되며, 결정론적 샘플링이 된다.

> ⚠️ **생성에 1000 스텝이 필요** — 이것이 DDPM의 가장 큰 단점이며, DDIM, Consistency Model 등 후속 연구의 동기. 256x256 이미지 하나를 생성하는 데 A100 GPU에서도 약 10초가 소요된다. 반면 GAN은 한 번의 forward pass로 생성하므로 수 밀리초면 충분하다.

---

5. 네트워크 구조: U-Net

5.1 왜 U-Net인가?

> 💡 **네트워크 선택의 동기** DDPM에서 뉴럴넷 $\epsilon_\theta(x_t, t)$의 역할은 명확하다: 노이즈가 추가된 이미지 x_t를 입력으로 받아, 거기에 포함된 노이즈 ϵ을 예측하는 것이다. 이때 핵심적인 구조적 요구사항이 있다: **입력과 출력의 크기가 동일**해야 한다 (둘 다 이미지와 같은 차원). 이것은 분류 네트워크 (입력: 이미지, 출력: 클래스 확률 벡터)와 근본적으로 다른 요구사항이다.

$\epsilon_\theta(x_t, t)$는 입력(x_t, 노이즈 이미지)과 **같은 크기의 출력**(예측 노이즈)을 반환해야 한다. 이미지-to-이미지 변환에 최적화된 U-Net이 자연스러운 선택.

U-Net (Ronneberger et al., 2015)은 원래 의료 이미지 분할(segmentation)을 위해 설계된 네트워크이다. "U"자 형태의 인코더-디코더 구조에 skip connection을 추가한 것이 특징이며, "입력 이미지와 같은 크기의 출력을 생성"하는 데 탁월하다. DDPM에서 U-Net을 선택한 이유는 (1) 인코더의 다운샘플링이 다양한 스케일의 특징을 포착하고, (2) skip connection이 고해상도 디테일을 보존하며, (3) 디코더가 이를 결합하여 정확한 픽셀 수준의 예측을 생성하기 때문이다. 특히 노이즈 예측에서 "전역적 구조 이해" (큰 t에서 중요)와 "지역적 디테일 복원" (작은 t에서 중요)을 동시에 수행해야 하므로, 다중 스케일 처리가 가능한 U-Net이 이상적이다.

5.2 구조

```
입력: x_t (노이즈 이미지) + t (타임스텝)

[Time Embedding]
  t -> Sinusoidal Encoding -> MLP -> t_emb (시간 조건 벡터)

[Encoder]                          [Decoder]
 ┌──────────┐                      ┌──────────┐
 │ ResBlock │──── skip ──────────>│ ResBlock │
 │ + t_emb  │                      │ + t_emb  │
 │ ↓ Down   │                      │ ↑ Up     │
 ├──────────┤                      ├──────────┤
 │ ResBlock │──── skip ──────────>│ ResBlock │
 │ + Attn   │                      │ + Attn   │
 │ ↓ Down   │                      │ ↑ Up     │
 ├──────────┤                      ├──────────┤
 │ ResBlock │──── skip ──────────>│ ResBlock │
 │ + Attn   │                      │ + Attn   │
 │ ↓ Down   │                      │ ↑ Up     │
 └──────────┘                      └──────────┘
       │                                 │
       └──────> [Bottleneck: ResBlock + Attn] ──┘

출력: epsilon_hat (예측 노이즈, 입력과 동일 크기)
```

5.3 핵심 구성 요소

(1) Time Embedding — 타임스텝을 모델에 주입

Transformer의 Positional Encoding과 동일한 sinusoidal 방식:

$$\mathrm{TE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \mathrm{TE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

이것을 MLP로 변환하여 각 ResBlock에 더한다:

```
t_emb = MLP(sinusoidal_embedding(t))  # [B, d_model]
# 각 ResBlock에서:
h = h + t_emb.unsqueeze(-1).unsqueeze(-1)  # [B, C, H, W]에 더하기
```

> 💡 **왜 t를 주입하는가?** 같은 모델이 $t = 1$부터 $t = 1000$까지 모든 노이즈 수준을 처리해야 하므로, 현재 t가 몇인지 알려줘야 한다. t를 알려주지 않으면 뉴럴넷은 입력 이미지 x_t만 보고 노이즈 수준을 추측해야 하는데, 이것은 불필요한 추가 부담이다. $t = 10$ (거의 깨끗한 이미지)에서의 노이즈 예측과 $t = 990$ (거의 순수 노이즈)에서의 노이즈 예측은 완전히 다른 성격의 작업이며, t를 명시적으로 주입함으로써 모델이 상황에 맞는 예측을 할 수 있게 된다.
>
> Sinusoidal Encoding을 사용하는 이유: 정수 t를 그대로 넣으면 (1) 스케일 문제 ($t = 1$과 $t = 1000$의 크기 차이가 크다), (2) 연속적 표현 부족 (인접한 t 값들 사이의 부드러운 전환이 어렵다) 등의 문제가 있다. Sinusoidal Encoding은 각 주파수 성분이 서로 다른 시간 스케일을 포착하여, t의 풍부한 표현을 제공한다. 이후 MLP가 이를 비선형 변환하여 모델에 최적화된 시간 조건 벡터로 만든다.

(2) Skip Connection

인코더의 고해상도 특징을 디코더에 직접 전달:

- 인코더: $64 \to 32 \to 16$ (해상도가 줄어들며 추상화)
- 디코더: $16 \to 32 \to 64$ (해상도를 복원하면서 스킵으로 디테일 보충)

Skip Connection이 없으면 디코더는 병목(bottleneck)의 저해상도 특징만으로 고해상도 디테일을 복원해야 하며, 이는 정보 손실로 인해 흐릿한 결과를 초래한다. Skip Connection은 인코더가 추출한 고해상도 특징 (가장자리, 텍스처 등)을 디코더에 직접 전달하여 이 문제를 해결한다. DDPM에서 이것은 특히 중요하다 — 노이즈 예측은 픽셀 수준의 정밀도를 요구하기 때문이다.

(3) Self-Attention

중간 해상도(16x16 또는 32x32)에서만 적용:

- 이유: Attention의 계산량이 $O(n^2)$ (n은 토큰 수 = 높이 $\times$ 너비)이므로 높은 해상도에서는 비용이 과다. 예: 64x64 해상도에서 attention은 4096x4096 행렬 연산이지만, 16x16에서는 256x256으로 약 256배 작다.
- 역할: 이미지 내 글로벌 관계 모델링. Convolution은 지역적 패턴만 포착하지만, Self-Attention은 이미지 전체에 걸친 장거리 의존성을 모델링한다. 예를 들어, 얼굴의 왼쪽 눈과 오른쪽 눈이 대칭이어야 한다는 관계는 Convolution만으로는 포착하기 어렵지만 Attention은 직접 포착할 수 있다.

> 💡 **DDPM U-Net과 일반 U-Net의 차이** DDPM의 U-Net은 원래의 의료 영상 U-Net과 비교하여 두 가지 핵심적 차이가 있다: (1) **시간 조건부(time-conditioned)**: 모든 ResBlock에 타임스텝 정보 t가 주입된다. (2) **Self-Attention 레이어**: 중간 해상도에서 글로벌 일관성을 위한 attention이 추가된다. 이 두 가지 추가가 DDPM의 높은 이미지 품질의 주요 요인이다.

---

6. Score-Based Model과의 관계

> 💡 **왜 Score-Based Model과의 관계가 중요한가?** DDPM과 Score-Based Model (강의 5)은 완전히 다른 동기에서 출발했다: DDPM은 VAE의 변분 추론 프레임워크를 확장한 것이고, Score-Based Model은 데이터 분포의 그래디언트(score)를 직접 학습하는 것이다. 그런데 놀랍게도, 두 접근법이 **수학적으로 완전히 동치**라는 것이 밝혀졌다 (Song et al., 2021). 이 통합적 이해가 중요한 이유는: (1) 두 분야의 이론적 도구를 서로 교차 사용할 수 있고, (2) SDE/ODE 프레임워크를 통해 더 나은 샘플링 방법 (DDIM, DPM-Solver 등)을 설계할 수 있으며, (3) 궁극적으로 Flow Matching 등 더 진보된 생성 모델로의 발전을 이끌었기 때문이다.

6.1 수학적 동치

강의 5에서 배운 **스코어 함수** (score function)와 DDPM의 노이즈 예측은 변환 관계에 있다. 스코어 함수란 로그 확률 밀도의 그래디언트 $\nabla_x \log p(x)$로, 데이터 공간에서 "확률이 높아지는 방향"을 가리키는 벡터장(vector field)이다. 직관적으로, 스코어 함수는 "현재 위치 x에서 더 그럴듯한 데이터가 있는 방향"을 알려준다.

상세 유도: $q(x_t \mid x_0)$의 스코어를 단계별로 계산한다.

Step 1: $q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$이므로, 로그를 취하면:

$$\log q(x_t \mid x_0) = -\frac{d}{2}\log(2\pi(1-\bar{\alpha}_t)) - \frac{\|x_t - \sqrt{\bar{\alpha}_t}x_0\|^2}{2(1-\bar{\alpha}_t)}$$

여기서 d는 데이터의 차원이다. 첫 번째 항은 x_t에 무관한 상수이므로 그래디언트에 기여하지 않는다.

Step 2: x_t에 대해 그래디언트를 계산한다 ($\nabla_{x_t}\|x_t - a\|^2 = 2(x_t - a)$를 사용):

$$\nabla_{x_t} \log q(x_t \mid x_0) = \nabla_{x_t}\left[-\frac{\|x_t - \sqrt{\bar{\alpha}_t}x_0\|^2}{2(1-\bar{\alpha}_t)}\right] = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t}$$

Step 3: Forward Process의 닫힌 형태 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$에서 $x_t - \sqrt{\bar{\alpha}_t}x_0 = \sqrt{1-\bar{\alpha}_t}\epsilon$이므로:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\sqrt{1-\bar{\alpha}_t}\epsilon}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

Step 4: 뉴럴넷 ϵ_θ가 ϵ을 근사하므로, 스코어 근사 s_θ와의 관계:

$$\boxed{s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}}$$

이 결과의 직관적 의미: **"노이즈를 예측하는 것"과 "데이터 방향을 가리키는 것"은 동일**하다. 노이즈 ϵ은 데이터에서 멀어지는 방향을 나타내므로, $-\epsilon$은 데이터 쪽으로 돌아가는 방향이다. 스코어 함수 $\nabla_x \log p(x)$도 "확률이 높은 방향 = 데이터가 있는 방향"을 가리킨다. 따라서 둘은 스케일링 팩터 $1/\sqrt{1-\bar{\alpha}_t}$를 제외하면 동일한 정보를 담고 있다.

Score-Based Model	**DDPM**
스코어 $s(x) = \nabla_x \log p(x)$ 학습	노이즈 ϵ 예측
Score Matching Loss	Simple Loss $\lvert\epsilon - \epsilon_\theta\rvert^2$
Langevin Dynamics로 샘플링	Reverse Process로 샘플링
수학적으로 동일	**수학적으로 동일**

이 동치성을 더 깊이 이해하기 위해: Score-Based Model의 Denoising Score Matching Loss는 $\mathbb{E}\left[\|\nabla_{x_t}\log q(x_t \mid x_0) - s_\theta(x_t, t)\|^2\right]$이고, 위의 관계를 대입하면 $\mathbb{E}\left[\frac{1}{1-\bar{\alpha}_t}\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$이 된다. 이것은 DDPM의 VLB와 (가중치를 제외하면) 동일하다. 즉, DDPM의 Simple Loss를 최적화하는 것은 Score Matching을 수행하는 것과 같다.

6.2 세 가지 예측 대상의 동치

같은 모델을 다르게 파라미터화할 수 있다. 이 세 가지는 모두 동일한 정보를 담고 있으며, 서로 선형 변환으로 변환 가능하다:

예측 대상	이름	수식	대표 모델
ϵ	노이즈 예측	$\epsilon_\theta(x_t, t) \approx \epsilon$	DDPM
x_0	원본 예측	$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta}{\sqrt{\bar{\alpha}_t}}$	일부 개선 모델
v	속도 예측	$v_\theta = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1-\bar{\alpha}_t}x_0$	SD 2.x, Flow Matching

세 가지 모두 수학적으로 동치이며, 서로 변환 가능하다.

v-prediction (velocity prediction)에 대한 추가 설명: v는 Forward Process의 "속도"로 해석할 수 있다. $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$을 연속 시간으로 보면, x_t가 x_0에서 ϵ 방향으로 이동하는 궤적이며, v는 이 궤적의 접선 벡터(tangent vector)이다. v-prediction은 ϵ-prediction에 비해 수치적으로 더 안정적인 학습을 제공하는 것으로 알려져 있으며, 특히 $t \approx 0$ (SNR이 큰) 근처에서의 학습이 개선된다. Stable Diffusion 2.x에서 v-prediction으로 전환한 것은 이러한 실험적 이점 때문이다.

---

7. DDIM — 빠른 결정론적 샘플링

> 💡 **왜 DDIM이 필요한가?** DDPM의 이미지 품질은 탁월하지만, 실용적으로 치명적인 단점이 있다: **생성이 너무 느리다**. 이미지 한 장을 만드는 데 1000번의 뉴럴넷 forward pass가 필요하며, 이는 GAN의 1회 forward pass와 비교하면 1000배 느린 것이다. DDIM은 이 문제를 해결하기 위해 DDPM의 **확률적** 역과정을 **결정론적** 역과정으로 변환하여, 훨씬 적은 스텝으로도 고품질 생성을 가능하게 한다.

7.1 DDPM의 문제

$T = 1000$ 스텝 샘플링은 너무 느리다. 256x256 이미지 하나에 ~10초.

7.2 핵심 아이디어

> 📄 **논문**: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021

DDPM의 역과정에서 **확률적 노이즈 항을 줄이거나 제거**하면, 더 적은 스텝으로도 좋은 결과를 얻을 수 있다.

DDIM 업데이트 규칙의 유도

DDIM의 핵심 통찰: DDPM의 Forward Process는 마르코프 체인이지만, **동일한 marginal 분포** $q(x_t \mid x_0)$를 유지하면서도 **비-마르코프(non-Markovian)** Forward Process를 정의할 수 있다. 즉, $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$는 동일하게 유지하되, $q(x_{t-1} \mid x_t, x_0)$의 형태를 더 자유롭게 설정한다.

구체적으로, DDIM은 다음과 같은 비-마르코프 posterior를 정의한다:

$$q_\sigma(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1-\bar{\alpha}_t}},\ \sigma_t^2 I\right)$$

여기서 σ_t는 자유 파라미터이다. 이 분포가 잘 정의되어 있음을 확인할 수 있다: x_0와 x_t가 주어졌을 때, x_{t-1}의 평균은 (i) x_0 방향의 성분 $\sqrt{\bar{\alpha}_{t-1}}x_0$와 (ii) x_t에서 x_0를 뺀 "노이즈 방향" 성분의 합

이며, 분산 σ_t^2로 확률적 변동을 조절한다.

생성 시에는 x_0를 모르므로, 뉴럴넷의 예측 $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}$를 대입하면:

DDIM의 업데이트 규칙:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\, \underbrace{\epsilon_\theta(x_t, t)}_{\text{방향}} + \underbrace{\sigma_t \epsilon_t}_{\text{노이즈}}$$

이 업데이트 규칙의 세 항을 직관적으로 해석하면:

1. **Predicted x_0 항** ($\sqrt{\bar{\alpha}_{t-1}}\hat{x}_0$): 뉴럴넷이 예측한 "깨끗한 이미지"를 $t-1$ 스텝의 스케일로 조정한 것. 이것이 x_{t-1}의 "신호 성분"이다.
2. **방향 항** ($\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \epsilon_\theta$): x_t에서 $\hat{x}_0$를 뺀 방향, 즉 "노이즈가 있어야 할 방향"으로의 성분. 이것이 x_{t-1}의 "결정론적 노이즈 성분"이다.
3. **노이즈 항** ($\sigma_t \epsilon_t$): 추가적인 확률적 노이즈. $\sigma_t = 0$이면 이 항이 사라져 완전히 결정론적이 된다.

- $\sigma_t = 0$: **결정론적** 샘플링 (DDIM) — 같은 x_T → 항상 같은 x_0
- $\sigma_t = \sqrt{\tilde{\beta}_t}$: DDPM과 동일 (확률적)
- $0 < \sigma_t < \sqrt{\tilde{\beta}_t}$: 중간 수준의 확률성 (실용적으로 유용)

> 💡 **왜 DDIM이 결정론적인가?** $\sigma_t = 0$으로 설정하면 확률적 항 $\sigma_t \epsilon_t$가 완전히 사라진다. 따라서 x_{t-1}은 x_t와 $\epsilon_\theta(x_t, t)$의 결정론적 함수가 된다. 초기 노이즈 x_T가 같으면 전체 경로 $x_T \to x_{T-1} \to \cdots \to x_0$가 완전히 결정되므로, 항상 같은 이미지 x_0가 생성된다. 이 결정론적 성질은 실용적으로 매우 유용하다: (1) 결과의 재현성(reproducibility), (2) 잠재 공간에서의 보간(interpolation) — 두 x_T 사이를 선형 보간하면 의미 있는 이미지 모핑이 가능하다.

> 💡 **왜 결정론적이면 더 적은 스텝이 가능한가?** DDPM의 확률적 역과정에서 각 스텝마다 추가되는 노이즈 $\sigma_t z$는 "경로를 흔드는" 역할을 한다. 큰 스텝을 뛰면 이 확률적 흔들림이 누적되어 결과가 불안정해진다. 반면 DDIM의 결정론적 역과정은 부드러운 곡선을 따라가므로, 큰 스텝으로 뛰어도 경로에서 크게 벗어나지 않는다. ODE 솔버의 관점에서 이해하면: 매끄러운 ODE 궤적은 큰 step size로도 정확하게 근사할 수 있지만, SDE 경로는 본질적으로 거친(rough) 경로이므로 작은 step size가 필요하다.

7.3 서브시퀀스 트릭

$\{1, 2, \ldots, 1000\}$ 전체가 아니라, 부분 시퀀스만 사용:

$$\tau = \{1, 51, 101, 151, \ldots, 951\} \quad (20 \text{ 스텝})$$

$T = 1000$에서 학습된 모델을 **그대로 사용**하면서 20~50 스텝으로 생성. 품질은 약간 떨어지지만 **50배 빠름**.

이것이 가능한 이유: DDIM의 업데이트 규칙은 $\bar{\alpha}_t$와 $\bar{\alpha}_{t-1}$의 값만 사용하고, 중간 스텝의 $\bar{\alpha}$ 값은 필요 없다. 따라서 $t = 951 \to t = 901$로 직접 점프할 때, $\bar{\alpha}_{951}$과 $\bar{\alpha}_{901}$만 알면 된다. 모델은 여전히 $\epsilon_\theta(x_t, t)$를 호출하되, t 값만 서브시퀀스의 원소를 사용한다.

7.4 DDIM은 ODE 솔버

$\sigma_t = 0$으로 설정한 DDIM은 Yang Song의 **Probability Flow ODE**를 Euler 방법으로 푸는 것과 동일하다.

Probability Flow ODE란 Forward SDE와 동일한 marginal 분포 $p_t(x)$를 생성하는 결정론적 ODE이다:

$$\frac{dx}{dt} = f(x,t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x)$$

여기서 $f(x,t)$는 Forward SDE의 drift 항이고, $g(t)$는 diffusion 계수이며, $\nabla_x \log p_t(x)$는 시간 t에서의 스코어 함수이다. 이 ODE의 의미: SDE의 확률적 경로를 "평균화"한 결정론적 경로로, 같은 분포를 생성하지만 노이즈 없이 매끄럽게 진행한다.

DDIM ($\sigma_t = 0$)의 업데이트 규칙은 이 ODE를 이산화한 **Euler method**에 해당한다. 따라서 더 정교한 ODE 솔버 (예: Heun method, Runge-Kutta)를 사용하면 더 적은 스텝으로 더 정확한 결과를 얻을 수 있다 — 이것이 DPM-Solver, DPM-Solver++ 등 후속 연구의 기반이다.

> DDIM이 결정론적(같은 노이즈 → 같은 이미지)인 이유: ODE는 결정론적이니까.
>
> 이 관찰이 Flow Matching으로의 발전을 이끌었다: "처음부터 ODE로 설계하면 어떨까?"

---

8. 노이즈 스케줄 설계

> 💡 **왜 노이즈 스케줄이 중요한가?** 노이즈 스케줄 $\{\beta_1, \ldots, \beta_T\}$는 Forward Process에서 노이즈가 추가되는 속도를 결정한다. 이것은 단순한 하이퍼파라미터가 아니라, DDPM의 성능에 직접적으로 영향을 미치는 핵심 설계 요소이다. 스케줄이 너무 빠르면 (큰 β_t) 이미지 정보가 일찍 소멸하여 뉴럴넷이 복원할 단서가 부족해지고, 너무 느리면 (작은 β_t) T 스텝 후에도 순수 노이즈에 도달하지 못해 생성 시작점 $\mathcal{N}(0, I)$과의 불일치가 발생한다. 좋은 스케줄은 T 스텝에 걸쳐 정보가 **균일하게** 소멸하여, 모든 t에서 뉴럴넷이 의미 있는 학습을 할 수 있게 한다.

8.1 선형 스케줄 (DDPM 원논문)

$$\beta_t = \beta_1 + \frac{t-1}{T-1}(\beta_T - \beta_1), \quad \beta_1 = 10^{-4},\ \beta_T = 0.02$$

선형 스케줄에서 β_t는 10^{-4}에서 0.02까지 균일하게 증가한다. 이에 따라 $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$는 초반에는 천천히 감소하다가 후반에 급격히 감소한다 (누적곱의 특성상 후반부에서 더 빠르게 0에 수렴). 이것은 Forward Process에서 "마지막 수백 스텝에서 정보가 급격히 소멸"한다는 의미이며, 이 구간에서 뉴럴넷이 학습할 수 있는 유의미한 정보가 거의 없다는 문제가 있다.

8.2 코사인 스케줄 (Improved DDPM)

> 📄 **논문**: Nichol & Dhariwal, "Improved Denoising Diffusion Probabilistic Models", ICML 2021

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)^2$$

여기서 $s = 0.008$은 작은 오프셋으로, $t = 0$ 근처에서 β_t가 너무 작아지는 것을 방지한다.

코사인 스케줄의 설계 철학: $\bar{\alpha}_t$를 직접 정의한다 (β_t에서 간접적으로 유도하는 것이 아니라). 코사인 함수의 제곱은 $t = 0$에서 1, $t = T$에서 0으로 **부드럽게 감소**한다. 이것은 선형 스케줄의 문제

— 끝부분에서 $\bar{\alpha}_t$가 너무 급격히 0으로 떨어져 정보 손실이 과도한 것 — 를 해결한다. 코사인 스케줄은 더 부드러운 감소를 제공하여, 전체 T 스텝에 걸쳐 정보가 더 균일하게 소멸한다.

실질적 효과: 코사인 스케줄을 사용하면 높은 t 값에서도 약간의 이미지 구조가 남아 있어, 뉴럴넷이 모든 t에서 의미 있는 학습을 할 수 있다. 이것은 특히 log-likelihood 개선에 효과적이다.

8.3 실용적 선택

스케줄	장점	단점	사용 모델
Linear	단순	끝부분 급격	DDPM 원본
Cosine	부드러운 감소	고해상도에서 비효율	Improved DDPM
Scaled Linear	잠재 공간에 최적화	—	Stable Diffusion

Scaled Linear에 대한 추가 설명: Stable Diffusion은 픽셀 공간이 아닌 **잠재 공간(latent space)**에서 Diffusion을 수행한다 (Latent Diffusion). 잠재 공간의 통계적 특성이 픽셀 공간과 다르기 때문에 (일반적으로 분산이 더 작다), β 범위를 $\sqrt{\text{scale}}$로 조정한 Scaled Linear 스케줄을 사용한다. 구체적으로 $\beta_{\text{start}} = 0.00085$, $\beta_{\text{end}} = 0.012$로, 원래 DDPM보다 좁은 범위를 사용한다.

---

9. 실습 가이드: PyTorch 구현

> 💡 **이론에서 구현으로** 지금까지 배운 수학적 내용을 실제 코드로 옮기는 과정이다. 코드를 읽을 때 "이 줄이 어떤 수식에 대응하는가?"를 항상 생각하며 읽어야 한다. 특히 핵심적인 대응 관계는 다음과 같다:
>
> - `NoiseSchedule` 클래스: Section 1의 $\alpha_t, \bar{\alpha}_t, \tilde{\beta}_t$ 등 모든 계수를 미리 계산
> - `forward_diffusion` 함수: Section 1.3의 닫힌 형태 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
> - Training Loop의 Loss: Section 3.4의 Simple Loss $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$
> - `ddpm_sample` 함수: Section 4.2의 Algorithm 2
> - `ddim_sample` 함수: Section 7.2의 DDIM 업데이트 규칙

> ⚠️ **참고**: 아래 코드는 교육 목적의 구현이다. 실제 프로덕션에서는 `diffusers` 라이브러리 등을 사용하는 것이 권장된다. 코드 전체를 하나의 스크립트로 합치면 CIFAR-10 등 작은 데이터셋에서 실행 가능하다.

9.1 NoiseSchedule 클래스

노이즈 스케줄의 모든 계수를 미리 계산하여 저장하는 클래스:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


class NoiseSchedule:
    """DDPM 노이즈 스케줄: beta, alpha, alpha_bar 등을 미리 계산.

    Args:
        T: 총 타임스텝 수 (기본 1000)
        beta_start: 시작 beta 값
        beta_end: 끝 beta 값
        schedule_type: 'linear' 또는 'cosine'
        device: torch device
    """
    def __init__(self, T=1000, beta_start=1e-4, beta_end=0.02,
                 schedule_type='linear', device='cuda'):
        self.T = T
        self.device = device

        if schedule_type == 'linear':
            # 선형 스케줄: beta_1 에서 beta_T 까지 균등 간격
            self.beta = torch.linspace(beta_start, beta_end, T, device=device
        elif schedule_type == 'cosine':
            # 코사인 스케줄 (Improved DDPM)
            s = 0.008  # 작은 offset으로 t=0 근처에서 beta가 너무 작아지는 것 방
            steps = torch.arange(T + 1, dtype=torch.float64, device=device)
            f_t = torch.cos(((steps / T) + s) / (1 + s) * (math.pi / 2)) ** 2
            alpha_bar = f_t / f_t[0]
            # beta_t = 1 - alpha_bar_t / alpha_bar_{t-1}, clamp으로 안정화
            beta = 1.0 - (alpha_bar[1:] / alpha_bar[:-1])
            self.beta = torch.clamp(beta, min=1e-4, max=0.999).float()
        else:
            raise ValueError(f"Unknown schedule type: {schedule_type}")

        # alpha_t = 1 - beta_t
        self.alpha = 1.0 - self.beta

        # alpha_bar_t = prod_{s=1}^{t} alpha_s  (누적곱)
        self.alpha_bar = torch.cumprod(self.alpha, dim=0)

        # alpha_bar_{t-1}: t=0일 때는 1.0 (아직 노이즈가 추가되지 않은 상태)
        self.alpha_bar_prev = F.pad(self.alpha_bar[:-1], (1, 0), value=1.0)

        # Posterior 분산: tilde_beta_t = (1 - alpha_bar_{t-1}) * beta_t / (1
        self.posterior_variance = (
            (1.0 - self.alpha_bar_prev) * self.beta / (1.0 - self.alpha_bar)
        )

        # sqrt 값들은 자주 쓰이므로 미리 계산
        self.sqrt_alpha = torch.sqrt(self.alpha)
        self.sqrt_alpha_bar = torch.sqrt(self.alpha_bar)
        self.sqrt_one_minus_alpha_bar = torch.sqrt(1.0 - self.alpha_bar)
        self.sqrt_recip_alpha = 1.0 / self.sqrt_alpha  # 1/sqrt(alpha_t)

    def get(self, values, t, x_shape):
        """t에 해당하는 값을 꺼내서 [B, 1, 1, 1] 형태로
```

```
    """t에 해당하는 값을 꺼내서 [B, 1, 1, 1] 형태로 reshape.

    이미지 텐서와 broadcasting하기 위해 필요한 유틸리티.

    Args:
        values: [T] 형태의 텐서 (예: self.alpha_bar)
        t: [B] 형태의 타임스텝 인덱스
        x_shape: 이미지 텐서의 shape (e.g., [B, C, H, W])
    Returns:
        [B, 1, 1, 1] 형태의 텐서
    """
    batch_size = t.shape[0]
    out = values.gather(0, t)  # [B]
    # [B] -> [B, 1, 1, 1] (이미지 차원에 맞게)
    return out.view(batch_size, *((1,) * (len(x_shape) - 1)))
```

9.2 Time Embedding

Sinusoidal Positional Encoding + MLP로 타임스텝을 벡터로 변환:

```python
class SinusoidalTimeEmbedding(nn.Module):
    """Sinusoidal 위치 인코딩으로 정수 타임스텝을 벡터로 변환.

    Transformer의 Positional Encoding과 동일한 방식.

    Args:
        dim: 임베딩 차원 (짝수여야 함)
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        """
        Args:
            t: [B] 형태의 정수 타임스텝 텐서
        Returns:
            [B, dim] 형태의 임베딩 벡터
        """
        device = t.device
        half_dim = self.dim // 2
        # log(10000) / (d/2 - 1) 을 미리 계산
        emb_scale = math.log(10000.0) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb_scale)
        # t[:, None] * emb[None, :] -> [B, half_dim]
        emb = t.float().unsqueeze(1) * emb.unsqueeze(0)
        # sin과 cos을 concat -> [B, dim]
        emb = torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1)
        return emb


class TimeMLPEmbedding(nn.Module):
    """Sinusoidal 인코딩 + 2-layer MLP로 시간 조건 벡터 생성.

    Args:
        time_dim: sinusoidal 인코딩 차원
        emb_dim: 출력 임베딩 차원 (보통 모델 채널 수의 4배)
    """
    def __init__(self, time_dim, emb_dim):
        super().__init__()
        self.sinusoidal = SinusoidalTimeEmbedding(time_dim)
        self.mlp = nn.Sequential(
            nn.Linear(time_dim, emb_dim),
            nn.SiLU(),                      # SiLU (Swish) 활성화 함수
            nn.Linear(emb_dim, emb_dim),
        )

    def forward(self, t):
        """
        Args:
            t: [B] 형태의 정수 타임스텝
        Returns:
            [B, emb_dim] 형태의 시간 조건 벡터
        """
        return self.mlp(self.sinusoidal(t))
```

9.3 ResBlock (시간 조건부)

시간 임베딩을 주입받는 Residual Block:

```python
class ResBlock(nn.Module):
    """시간 조건부 Residual Block.

    구조: GroupNorm -> SiLU -> Conv -> (+ time_emb) -> GroupNorm -> SiLU -> D

    Args:
        in_ch: 입력 채널 수
        out_ch: 출력 채널 수
        time_emb_dim: 시간 임베딩 벡터 차원
        dropout: 드롭아웃 비율
    """
    def __init__(self, in_ch, out_ch, time_emb_dim, dropout=0.1):
        super().__init__()
        # 첫 번째 convolution 경로
        self.norm1 = nn.GroupNorm(32, in_ch)
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)

        # 시간 임베딩 projection: emb_dim -> out_ch
        self.time_proj = nn.Sequential(
            nn.SiLU(),
            nn.Linear(time_emb_dim, out_ch),
        )

        # 두 번째 convolution 경로
        self.norm2 = nn.GroupNorm(32, out_ch)
        self.dropout = nn.Dropout(dropout)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)

        # Skip connection: 채널 수가 다르면 1x1 conv로 맞춤
        if in_ch != out_ch:
            self.skip_conv = nn.Conv2d(in_ch, out_ch, kernel_size=1)
        else:
            self.skip_conv = nn.Identity()

    def forward(self, x, t_emb):
        """
        Args:
            x: [B, in_ch, H, W] 입력 특징맵
            t_emb: [B, time_emb_dim] 시간 조건 벡터
        Returns:
            [B, out_ch, H, W] 출력 특징맵
        """
        h = self.norm1(x)
        h = F.silu(h)
        h = self.conv1(h)

        # 시간 임베딩 주입: [B, out_ch] -> [B, out_ch, 1, 1]
        t = self.time_proj(t_emb)[:, :, None, None]
        h = h + t  # broadcasting으로 더하기

        h = self.norm2(h)
        h = F.silu(h)
        h = self.dropout(h)
        h = self.conv2(h)

        # Residual connection
        return h + self.skip_conv(x)
```

9.4 Self-Attention 블록

중간 해상도에서 글로벌 관계를 포착하기 위한 Self-Attention:

```python
class SelfAttention(nn.Module):
    """간단한 Multi-Head Self-Attention 블록.

    이미지 특징맵을 시퀀스로 변환하여 attention 수행 후 다시 이미지 형태로 복원.

    Args:
        channels: 입력/출력 채널 수
        num_heads: attention head 수 (기본 4)
    """
    def __init__(self, channels, num_heads=4):
        super().__init__()
        self.channels = channels
        self.num_heads = num_heads
        self.head_dim = channels // num_heads

        self.norm = nn.GroupNorm(32, channels)
        self.qkv = nn.Conv2d(channels, channels * 3, kernel_size=1)
        self.proj_out = nn.Conv2d(channels, channels, kernel_size=1)

    def forward(self, x):
        """
        Args:
            x: [B, C, H, W]
        Returns:
            [B, C, H, W] (residual connection 포함)
        """
        B, C, H, W = x.shape
        h = self.norm(x)

        # Q, K, V 계산: [B, 3C, H, W] -> 3 * [B, C, H, W]
        qkv = self.qkv(h)
        qkv = qkv.reshape(B, 3, self.num_heads, self.head_dim, H * W)
        qkv = qkv.permute(1, 0, 2, 4, 3)  # [3, B, heads, HW, head_dim]
        q, k, v = qkv[0], qkv[1], qkv[2]

        # Scaled Dot-Product Attention
        scale = self.head_dim ** -0.5
        attn = torch.matmul(q, k.transpose(-2, -1)) * scale  # [B, heads, HW,
        attn = F.softmax(attn, dim=-1)

        # Attention 적용
        out = torch.matmul(attn, v)  # [B, heads, HW, head_dim]
        out = out.permute(0, 1, 3, 2).reshape(B, C, H, W)

        return x + self.proj_out(out)  # Residual connection
```

9.5 U-Net 전체 구조

위 구성 요소를 조합한 전체 U-Net:

```python
class Downsample(nn.Module):
    """2x 다운샘플링 (stride-2 convolution)."""
    def __init__(self, channels):
        super().__init__()
        self.conv = nn.Conv2d(channels, channels, kernel_size=3, stride=2, pa

    def forward(self, x):
        return self.conv(x)


class Upsample(nn.Module):
    """2x 업샘플링 (nearest interpolation + convolution)."""
    def __init__(self, channels):
        super().__init__()
        self.conv = nn.Conv2d(channels, channels, kernel_size=3, padding=1)

    def forward(self, x):
        x = F.interpolate(x, scale_factor=2, mode='nearest')
        return self.conv(x)


class UNet(nn.Module):
    """DDPM U-Net: 시간 조건부 노이즈 예측 네트워크.

    Args:
        in_channels: 입력 이미지 채널 (예: RGB=3)
        model_channels: 기본 채널 수 (기본 128)
        out_channels: 출력 채널 (보통 in_channels와 동일)
        channel_mult: 각 해상도 레벨의 채널 배수 (예: (1, 2, 2, 4))
        attn_resolutions: attention을 적용할 다운샘플링 레벨 인덱스
        num_res_blocks: 각 레벨당 ResBlock 수
        dropout: 드롭아웃 비율
    """
    def __init__(
        self,
        in_channels=3,
        model_channels=128,
        out_channels=3,
        channel_mult=(1, 2, 2, 4),
        attn_resolutions=(1, 2),   # 두 번째, 세 번째 레벨에서 attention
        num_res_blocks=2,
        dropout=0.1,
    ):
        super().__init__()
        time_emb_dim = model_channels * 4

        # --- 시간 임베딩 ---
        self.time_embed = TimeMLPEmbedding(model_channels, time_emb_dim)

        # --- 입력 projection ---
        self.input_conv = nn.Conv2d(in_channels, model_channels, kernel_size=

        # --- 인코더 (Downsampling path) ---
        self.down_blocks = nn.ModuleList()
        self.down_samples = nn.ModuleList()
        channels = [model_channels]
        ch = model_channels
```

```
        for level, mult in enumerate(channel_mult):
            out_ch = model_channels * mult
            for _ in range(num_res_blocks):
                layers = [ResBlock(ch, out_ch, time_emb_dim, dropout)]
                if level in attn_resolutions:
                    layers.append(SelfAttention(out_ch))
                self.down_blocks.append(nn.ModuleList(layers))
                ch = out_ch
                channels.append(ch)
            # 마지막 레벨에서는 다운샘플링 하지 않음
            if level < len(channel_mult) - 1:
                self.down_samples.append(Downsample(ch))
                channels.append(ch)

        # --- 병목 (Bottleneck) ---
        self.mid_block1 = ResBlock(ch, ch, time_emb_dim, dropout)
        self.mid_attn = SelfAttention(ch)
        self.mid_block2 = ResBlock(ch, ch, time_emb_dim, dropout)

        # --- 디코더 (Upsampling path) ---
        self.up_blocks = nn.ModuleList()
        self.up_samples = nn.ModuleList()

        for level, mult in reversed(list(enumerate(channel_mult))):
            out_ch = model_channels * mult
            for i in range(num_res_blocks + 1):
                # skip connection에서 오는 채널을 concat
                skip_ch = channels.pop()
                layers = [ResBlock(ch + skip_ch, out_ch, time_emb_dim, dropou
                if level in attn_resolutions:
                    layers.append(SelfAttention(out_ch))
                self.up_blocks.append(nn.ModuleList(layers))
                ch = out_ch
            if level > 0:
                self.up_samples.append(Upsample(ch))

        # --- 출력 ---
        self.out_norm = nn.GroupNorm(32, ch)
        self.out_conv = nn.Conv2d(ch, out_channels, kernel_size=3, padding=1)

    def forward(self, x, t):
        """
        Args:
            x: [B, C, H, W] 노이즈가 추가된 이미지 x_t
            t: [B] 타임스텝 (정수)
        Returns:
            [B, C, H, W] 예측 노이즈 epsilon_hat
        """
        # 시간 임베딩
        t_emb = self.time_embed(t)  # [B, time_emb_dim]

        # 입력 projection
        h = self.input_conv(x)
        skip_connections = [h]

        # --- 인코더 ---
        ds idx = 0
```

```
        for i, block_layers in enumerate(self.down_blocks):
            res_block = block_layers[0]
            h = res_block(h, t_emb)
            if len(block_layers) > 1:
                attn_block = block_layers[1]
                h = attn_block(h)
            skip_connections.append(h)
            # 다운샘플링이 필요한 위치인지 확인
            # (num_res_blocks개 블록 후에 다운샘플링)

        # 다운샘플링은 별도로 처리
        # 간략화를 위해 실제 구현에서는 더 세밀한 인덱싱이 필요

        # --- 병목 ---
        h = self.mid_block1(h, t_emb)
        h = self.mid_attn(h)
        h = self.mid_block2(h, t_emb)

        # --- 디코더 ---
        for block_layers in self.up_blocks:
            res_block = block_layers[0]
            skip = skip_connections.pop()
            h = torch.cat([h, skip], dim=1)  # skip connection concat
            h = res_block(h, t_emb)
            if len(block_layers) > 1:
                attn_block = block_layers[1]
                h = attn_block(h)

        # --- 출력 ---
        h = self.out_norm(h)
        h = F.silu(h)
        return self.out_conv(h)
```

> ⚠️ **참고**: 위 U-Net은 구조를 이해하기 위한 교육용 코드이다. 실제 DDPM 논문의 구현은 다운/업샘플링 위치, skip connection 인덱싱 등이 더 세밀하게 처리된다. 완전한 구현은 lucidrains/denoising-diffusion-pytorch 또는 OpenAI의 guided-diffusion 을 참고하라.

9.6 Forward Diffusion

학습 시 x_0에서 x_t를 한 번에 생성하는 함수. 이것은 Section 1.3에서 유도한 닫힌 형태 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$을 직접 구현한 것이다. 핵심: ϵ은 "추가된 노이즈"이자 "학습 타겟"으로, 이 함수가 반환하는 ϵ이 바로 뉴럴넷이 예측해야 할 정답이다.

```python
def forward_diffusion(x_0, t, schedule):
    """Forward process: x_0에서 x_t를 한 번에 생성.

    수식: x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * epsilon

    Args:
        x_0: [B, C, H, W] 깨끗한 원본 이미지
        t: [B] 타임스텝 인덱스 (0-indexed: 0 ~ T-1)
        schedule: NoiseSchedule 인스턴스
    Returns:
        x_t: [B, C, H, W] 노이즈가 추가된 이미지
        epsilon: [B, C, H, W] 추가된 노이즈 (학습 타겟)
    """
    # 계수를 꺼내서 [B, 1, 1, 1] 형태로 reshape
    sqrt_alpha_bar = schedule.get(schedule.sqrt_alpha_bar, t, x_0.shape)
    sqrt_one_minus_alpha_bar = schedule.get(
        schedule.sqrt_one_minus_alpha_bar, t, x_0.shape
    )

    # 랜덤 가우시안 노이즈
    epsilon = torch.randn_like(x_0)

    # x_t = sqrt(alpha_bar) * x_0 + sqrt(1 - alpha_bar) * epsilon
    x_t = sqrt_alpha_bar * x_0 + sqrt_one_minus_alpha_bar * epsilon

    return x_t, epsilon
```

9.7 학습 루프 (Training Loop)

```python
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


def train_ddpm(
    model,
    schedule,
    dataloader,
    num_epochs=100,
    lr=2e-4,
    device='cuda',
    grad_clip=1.0,
):
    """DDPM 학습 루프.

    Args:
        model: U-Net 모델
        schedule: NoiseSchedule 인스턴스
        dataloader: 학습 데이터 DataLoader
        num_epochs: 학습 에폭 수
        lr: 학습률
        device: 'cuda' 또는 'cpu'
        grad_clip: gradient clipping 최대 norm
    """
    model = model.to(device)
    optimizer = optim.AdamW(model.parameters(), lr=lr)
    # EMA (Exponential Moving Average)는 생성 품질에 중요하지만,
    # 여기서는 핵심 알고리즘에 집중하기 위해 생략.

    model.train()
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        for batch_idx, (x_0, _) in enumerate(dataloader):
            x_0 = x_0.to(device)
            batch_size = x_0.shape[0]

            # 1. 랜덤 타임스텝 선택 (0-indexed: 0 ~ T-1)
            t = torch.randint(0, schedule.T, (batch_size,), device=device)

            # 2. Forward diffusion: x_0 -> x_t (한 번에)
            x_t, epsilon = forward_diffusion(x_0, t, schedule)

            # 3. 노이즈 예측
            epsilon_pred = model(x_t, t)

            # 4. Simple Loss: MSE(실제 노이즈, 예측 노이즈)
            loss = F.mse_loss(epsilon_pred, epsilon)

            # 5. 역전파 + 최적화
            optimizer.zero_grad()
            loss.backward()
            # Gradient clipping: 학습 안정화에 도움
            torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
            optimizer.step()

            epoch_loss += loss.item()
```

```
        avg_loss = epoch_loss / len(dataloader)
        print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.6f}")
```

> ⚠️ **EMA (Exponential Moving Average)**: DDPM 논문에서는 모델 파라미터의 EMA를 사용하여 생성 품질을 크게 향상시켰다. EMA decay rate는 보통 0.9999이며, 평가/생성 시에는 EMA 파라미터를 사용한다. 위 코드에서는 간결함을 위해 생략하였으나 실전에서는 반드시 포함해야 한다.

9.8 DDPM 샘플링 (생성)

학습된 모델로 순수 노이즈에서 이미지를 생성:

```python
@torch.no_grad()
def ddpm_sample(model, schedule, image_shape, device='cuda'):
    """DDPM 샘플링: 순수 노이즈에서 이미지 생성.

    Algorithm 2 (Sampling) 의 구현.
    T 스텝을 역순으로 진행하며 노이즈를 제거한다.

    Args:
        model: 학습된 U-Net 모델
        schedule: NoiseSchedule 인스턴스
        image_shape: 생성할 이미지 shape (B, C, H, W)
        device: torch device
    Returns:
        x: [B, C, H, W] 생성된 이미지
    """
    model.eval()
    B = image_shape[0]

    # x_T ~ N(0, I): 순수 가우시안 노이즈에서 시작
    x = torch.randn(image_shape, device=device)

    for t_idx in reversed(range(schedule.T)):
        # 현재 타임스텝을 배치 크기만큼 반복
        t = torch.full((B,), t_idx, device=device, dtype=torch.long)

        # 노이즈 예측
        eps_pred = model(x, t)

        # 계수 추출
        alpha = schedule.get(schedule.alpha, t, x.shape)
        alpha_bar = schedule.get(schedule.alpha_bar, t, x.shape)
        beta = schedule.get(schedule.beta, t, x.shape)
        sqrt_recip_alpha = schedule.get(schedule.sqrt_recip_alpha, t, x.shape
        sqrt_one_minus_alpha_bar = schedule.get(
            schedule.sqrt_one_minus_alpha_bar, t, x.shape
        )

        # 예측 평균: mu = (1/sqrt(alpha_t)) * (x_t - beta_t/sqrt(1-alpha_bar_t
        mean = sqrt_recip_alpha * (x - beta / sqrt_one_minus_alpha_bar * eps_

        if t_idx > 0:
            # t > 1: 노이즈 추가 (확률적 다양성)
            z = torch.randn_like(x)
            sigma = torch.sqrt(beta)  # 또는 posterior_variance 사용
            x = mean + sigma * z
        else:
            # t = 1 (마지막 스텝): 노이즈 추가하지 않음
            x = mean

    return x


# 사용 예시:
# images = ddpm_sample(model, schedule, (16, 3, 32, 32))
# 16장의 32x32 RGB 이미지 생성
```

9.9 DDIM 샘플링 (빠른 생성)

DDIM으로 더 적은 스텝에서 생성하는 코드:

```python
@torch.no_grad()
def ddim_sample(
    model, schedule, image_shape,
    ddim_steps=50, eta=0.0, device='cuda',
):
    """DDIM 샘플링: 서브시퀀스를 사용한 빠른 생성.

    Args:
        model: 학습된 U-Net 모델 (DDPM으로 학습한 모델 그대로 사용)
        schedule: NoiseSchedule 인스턴스
        image_shape: 생성할 이미지 shape (B, C, H, W)
        ddim_steps: DDIM 샘플링 스텝 수 (기본 50)
        eta: 확률성 조절 파라미터
             eta=0: 완전 결정론적 (DDIM)
             eta=1: DDPM과 동일 (확률적)
        device: torch device
    Returns:
        x: [B, C, H, W] 생성된 이미지
    """
    model.eval()
    B = image_shape[0]

    # 서브시퀀스 생성: T 스텝 중 ddim_steps 개만 균등 간격으로 선택
    # 예: T=1000, ddim_steps=50 -> [0, 20, 40, ..., 980]
    step_size = schedule.T // ddim_steps
    timesteps = list(range(0, schedule.T, step_size))
    timesteps = list(reversed(timesteps))  # 역순으로 진행

    # x_T ~ N(0, I)
    x = torch.randn(image_shape, device=device)

    for i in range(len(timesteps)):
        t_cur = timesteps[i]
        t_prev = timesteps[i + 1] if i + 1 < len(timesteps) else 0

        t = torch.full((B,), t_cur, device=device, dtype=torch.long)

        # 노이즈 예측
        eps_pred = model(x, t)

        # alpha_bar 값 추출
        alpha_bar_t = schedule.alpha_bar[t_cur]
        alpha_bar_t_prev = schedule.alpha_bar[t_prev] if t_prev > 0 else torc

        # predicted x_0
        x0_pred = (x - torch.sqrt(1 - alpha_bar_t) * eps_pred) / torch.sqrt(a
        # 선택적으로 x0_pred를 [-1, 1]로 clamp하면 품질이 향상될 수 있음
        x0_pred = torch.clamp(x0_pred, -1.0, 1.0)

        # DDIM 분산 계산
        sigma_t = eta * torch.sqrt(
            (1 - alpha_bar_t_prev) / (1 - alpha_bar_t) * (1 - alpha_bar_t / a
        )

        # "방향" (direction pointing to x_t)
        dir_xt = torch.sqrt(1 - alpha_bar_t_prev - sigma_t ** 2) * eps_pred

        # DDIM 업데이트
```

```
        # DDIM 업데이트
        x = torch.sqrt(alpha_bar_t_prev) * x0_pred + dir_xt

        # eta > 0이면 노이즈 추가
        if sigma_t > 0 and t_cur > 0:
            noise = torch.randn_like(x)
            x = x + sigma_t * noise

    return x


# 사용 예시:
# 결정론적 생성 (50 스텝, ~20배 빠름):
# images = ddim_sample(model, schedule, (16, 3, 32, 32), ddim_steps=50, eta=0
#
# 확률적 생성 (DDPM과 동일하지만 50 스텝):
# images = ddim_sample(model, schedule, (16, 3, 32, 32), ddim_steps=50, eta=1
```

---

10. 핵심 수식 요약

아래 표는 DDPM의 모든 핵심 수식을 한눈에 정리한 것이다. 각 수식이 어떤 섹션에서 유도되었는지를 참조하면 복습에 도움이 된다.

이름	수식	유도 위치
Forward (한 스텝)	$q(x_t\|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\,x_{t-1},\ \beta_t I)$	Section 1.1
Forward (닫힌 형태)	$q(x_t\|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,x_0,\ (1-\bar{\alpha}_t)I)$	Section 1.3
Posterior	$q(x_{t-1}\|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t,\ \tilde{\beta}_t I)$	Section 2.3
Posterior 평균	$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$	Section 2.4
Posterior 분산	$\tilde{\beta}_t = \frac{(1-\bar{\alpha}_{t-1})\beta_t}{1-\bar{\alpha}_t}$	Section 2.3
Simple Loss	$\mathcal{L} = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t,t)\|^2\right]$	Section 3.4
Score 관계	$s_\theta = -\epsilon_\theta/\sqrt{1-\bar{\alpha}_t}$	Section 6.1
DDIM 업데이트	$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\,\epsilon_\theta + \sigma_t\epsilon_t$	Section 7.2

핵심 표기법 정리: $\alpha_t = 1-\beta_t$, $\bar{\alpha}_t = \prod_{s=1}^{t}\alpha_s$, $\mathrm{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$.

---

11. 다음 강의 미리보기

DDPM의 두 가지 한계:

1. **1000 스텝 생성이 느리다** → DDIM, DPM-Solver, Consistency Model (강의 11)
2. **고해상도에서 너무 비싸다** → Latent Diffusion으로 해결 (강의 9)

다음 강의에서는 **CLIP**과 **조건부 생성(Classifier-Free Guidance)**을 배운 뒤, 이들을 결합한 **Latent Diffusion / Stable Diffusion**으로 진행한다.

이 강의에서 배운 DDPM은 **모든 현대 Diffusion 기반 생성 모델의 수학적 기초**이다. Stable Diffusion, DALL-E 2, Imagen, Midjourney 등은 모두 DDPM의 Forward/Reverse Process, ϵ-prediction, Simple Loss를 기반으로 하며, 여기에 잠재 공간 변환 (Latent Diffusion), 텍스트 조건부 생성 (Cross-Attention + CLIP), 가이던스 기법 (Classifier-Free Guidance) 등을 추가한 것이다. 따라서 이 강의의 내용을 완전히 이해하는 것이 후속 강의들의 필수 전제 조건이다.

---

12. 참고문헌 (References)

1. Ho, J., Jain, A., & Abbeel, P. (2020). **"Denoising Diffusion Probabilistic Models"**. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 6840-6851. arXiv:2006.11239
2. Song, J., Meng, C., & Ermon, S. (2021). **"Denoising Diffusion Implicit Models"**. *International Conference on Learning Representations (ICLR)*. arXiv:2010.02502
3. Nichol, A. Q., & Dhariwal, P. (2021). **"Improved Denoising Diffusion Probabilistic Models"**. *International Conference on Machine Learning (ICML)*. arXiv:2102.09672
4. Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). **"Score-Based Generative Modeling through Stochastic Differential Equations"**. *International Conference on Learning Representations (ICLR)*. arXiv:2011.13456
5. Ronneberger, O., Fischer, P., & Brox, T. (2015). **"U-Net: Convolutional Networks for Biomedical Image Segmentation"**. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. arXiv:1505.04597
6. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). **"Deep Unsupervised Learning using Nonequilibrium Thermodynamics"**. *International Conference on Machine Learning (ICML)*. arXiv:1503.03585
7. Kingma, D. P., & Welling, M. (2014). **"Auto-Encoding Variational Bayes"**. *International Conference on Learning Representations (ICLR)*. arXiv:1312.6114
8. Dhariwal, P., & Nichol, A. (2021). **"Diffusion Models Beat GANs on Image Synthesis"**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2105.05233